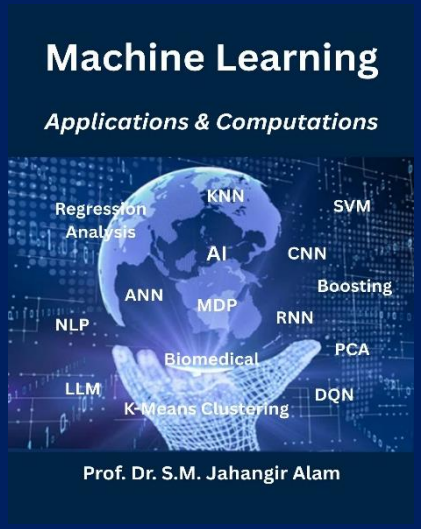
Lecture 3.1

Unsupervised Machine Learning

PCA (Principal Component Analysis)

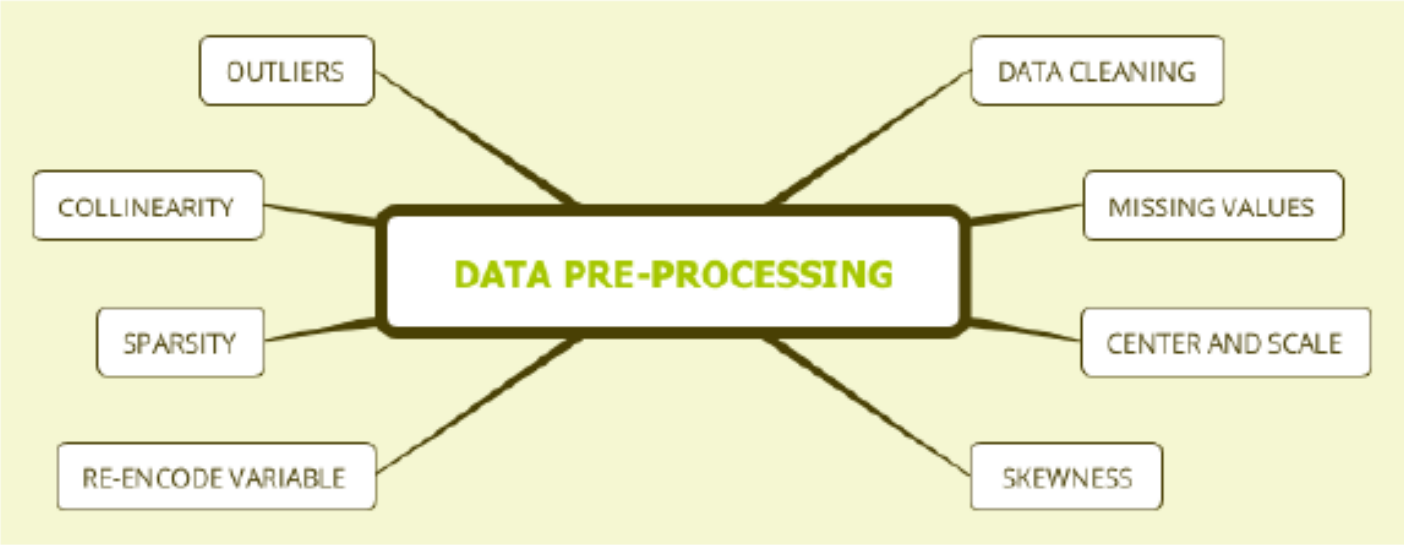


Instructor Name:

Professor Dr. Jahangir Alam

Professor, Dept. of CSE, BAUST

Data Pre-Processing



1. Raw data
2. Technically correct data
3. Data that is proper for the model
4. Summarized data
5. Data with fixed format

Centering and Scaling

- principle component analysis (PCA) (Jolliffe, 2002),
- partial least squares (PLS) (Geladi P,1986) and
- factor analysis (Mulaik, 2009).

PCA

- Principal Component Analysis (PCA) is an unsupervised linear dimensionality reduction technique that transforms high-dimensional data into a smaller number of uncorrelated variables, called principal components.

- It finds directions (axes) in which the data varies the most.

- Helps you find out the most common dimensions of your project and makes result analysis easier.

PCA finds:

. The **direction of maximum variance** in the data

. The **best lower-dimensional space** to represent it

→ It's used to **compress**, **visualize**, and **denoise** data.

Objectives of PCA

- Covariance between the new features (in case of PCA, they are the principal components) is 0.
- First principal component captures maximum variability, second PC capture next highest variability etc.
- Sum of the variance of the new features / the principal components = sum of variance of the original features.

Applications of PCA

- PCA — one of the most fundamental and widely used techniques in computer vision, pattern recognition, and image compression.
- Data cleaning and data preprocessing techniques.
- Monitor multi-dimensional data (can visualize in 2D or 3D dimensions) over any platform using the Principal Component Method of factor analysis.
- Compress the information and transmit the same using effective PCA analysis techniques.
- All techniques are without any loss in quality.

Applications of PCA

- Face recognition, image identification, pattern identification, and a lot more.
- PCA in machine learning technique helps in simplifying complex business algorithms
- PCA minimizes the more significant variance of dimensions, denoise information and completely omit noise and external factors.
- PCA is used to visualize multidimensional data.

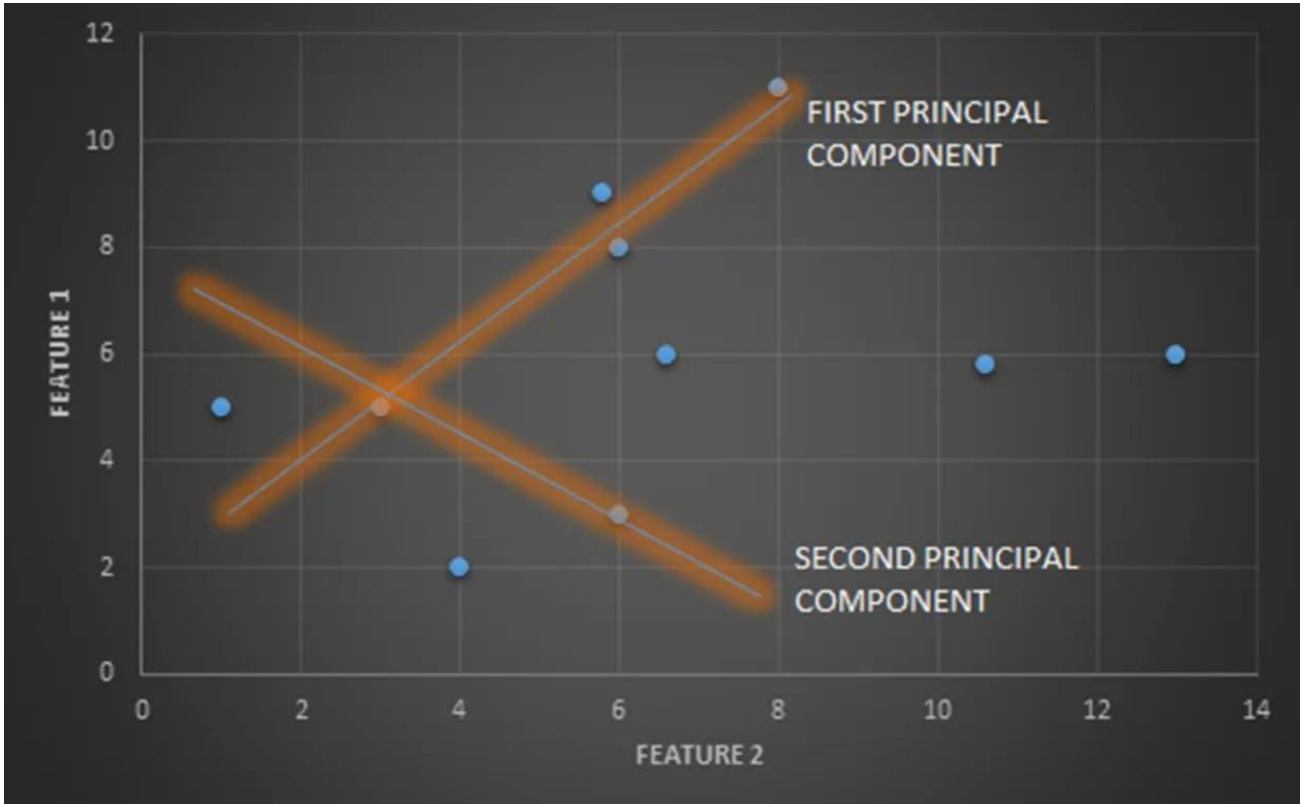
Applications of PCA

- In healthcare data, to explore the factors that are assumed to be very important in increasing the risk of any chronic disease.
- PCA helps to resize an image.
- PCA is used to analyze stock data and forecasting data.
- To analyze patterns when you are dealing with high-dimensional data sets.

Applications of PCA

Application	Purpose
Face Recognition (Eigenfaces)	Reduce face image dimensions
Image Compression	Keep only top principal components
Noise Reduction	Remove minor components (low variance = noise)
Object Recognition	Simplify high-dimensional feature sets
Medical Imaging (CT/MRI)	Analyze and compress diagnostic images
Remote Sensing	Reduce hyperspectral image bands

PCA



1st PC: similar or greater amount of variance are grouped and

2nd PC: varying or smaller variance are grouped.

PCA Algorithm

Step-01: Get data.

Step-02: Compute the mean vector (μ).

Step-03: Subtract mean from the given data.

Step-04: Calculate the covariance matrix.

Step-05: Calculate eigen vectors & eigen values of covariance matrix.

Step-06: Choosing components and forming a feature vector.

Step-07: Deriving the new data set.

PCA Algorithm

Let your data be X with n samples and d features.

Step 1: Mean Centering

$$X_{centered} = X - \mu$$

Subtract the mean from each feature.

Step 2: Covariance Matrix

$$C = \frac{1}{n-1} X_{centered}^{T} X_{centered}$$

Measures how features vary together.

Step 3: Eigen Decomposition

$$Cv_i = \lambda_i v_i$$

λ_i: Eigenvalue → variance captured

v_i: Eigenvector → principal direction

PCA Algorithm

Step 4: Sort by Variance

Sort eigenvectors by descending eigenvalues.
Top k eigenvectors = most significant directions.

Step 5: Project Data

$$Y = X_{centered} \cdot W$$

Where W = matrix of top k eigenvectors.

→ Y is the PCA-reduced representation.

***Note:**

Eigenvectors → "Eigenfaces" (directions of facial variation).

Eigenvalues → Importance (how much each direction explains data variation).

→ Reduces storage and speeds up recognition.

Example#1: PCA

Given data = { 2, 3, 4, 5, 6, 7 ; 1, 5, 3, 6, 7, 8 }.

Compute the principal component using PCA Algorithm.

OR

Consider the 2D patterns (2, 1), (3, 5), (4, 3), (5, 6), (6, 7), (7, 8).

Compute the principal component using PCA Algorithm.

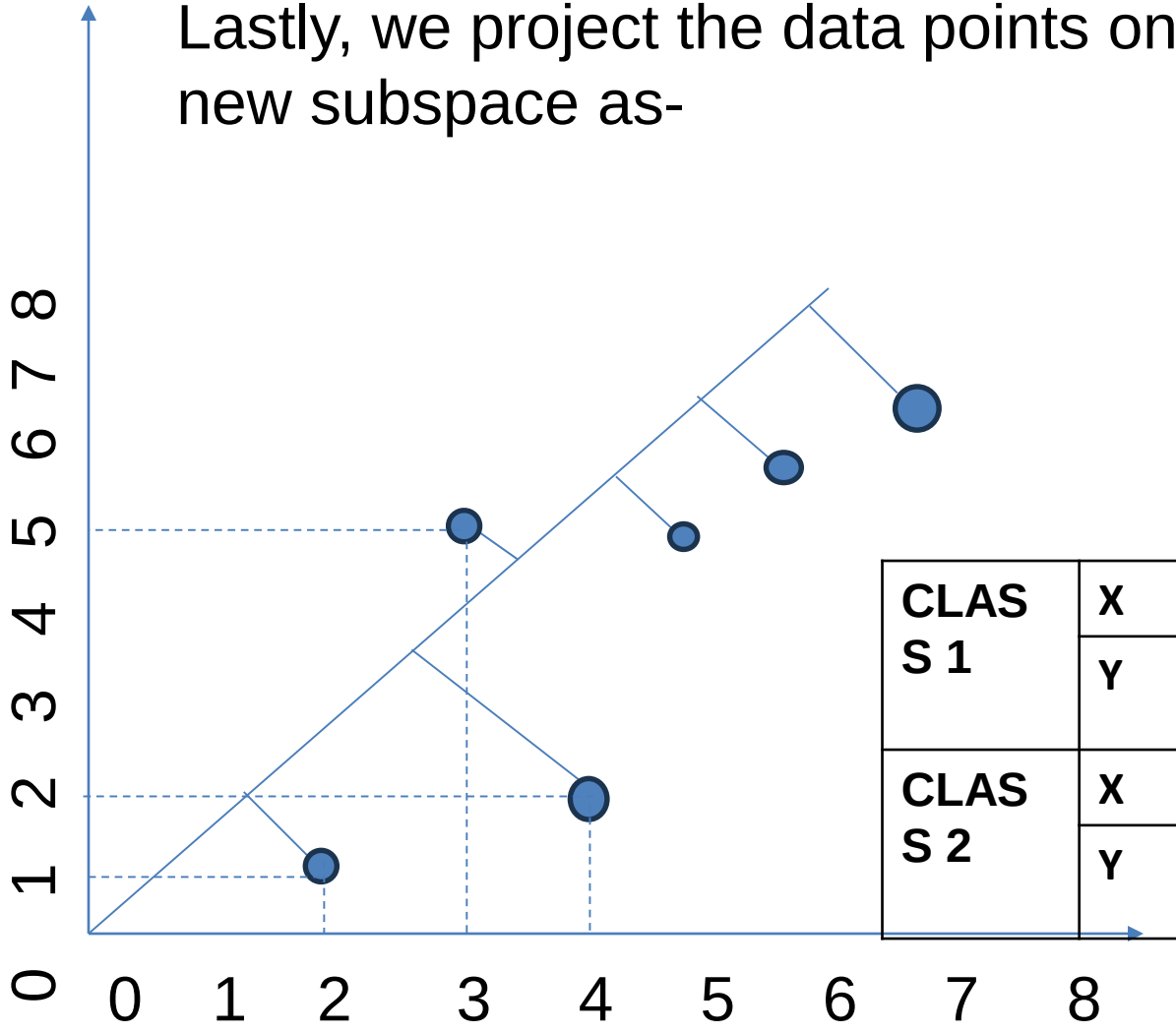
OR

Compute the principal component of following data-

CLASS 1	**X**	**2**	**3**	**4**
	Y	**1**	**5**	**3**
CLASS 2	**X**	**5**	**6**	**7**
	Y	**6**	**7**	**8**

Example#1: PCA

Lastly, we project the data points onto the new subspace as-



CLASS 1	X	2	3	4
	Y	1	5	3
CLASS 2	X	5	6	7
	Y	6	7	8

Example#1: PCA

1. Get data. The given feature vectors are-

X1	X2	X3	X4	X5	X6
(2,1)	(3,5)	(4,3)	(5,6)	(6,7)	(7,8)

$$\begin{bmatrix}2\\1\end{bmatrix} \quad \begin{bmatrix}3\\5\end{bmatrix} \quad \begin{bmatrix}4\\3\end{bmatrix} \quad \begin{bmatrix}5\\6\end{bmatrix} \quad \begin{bmatrix}6\\7\end{bmatrix} \quad \begin{bmatrix}7\\8\end{bmatrix}$$

2. Calculate the mean vector (μ).

μ = ((2 + 3 + 4 + 5 + 6 + 7) / 6, (1 + 5 + 3 + 6 + 7 + 8) / 6)= (4.5, 5)= $\begin{bmatrix}4.5\\5\end{bmatrix}$

3. Subtract mean vector (μ) from the given feature vectors.

X1-μ	X2-μ	X3-μ	X4-μ	X5-μ	X6-μ
(-2.5,-4)	(-1.5,0)	(-0.5,-2)	(0.5,1)	(1.5,2)	(2.5,3)

$$\begin{bmatrix}-2.5\\-4\end{bmatrix} \quad \begin{bmatrix}-1.5\\0\end{bmatrix} \quad \begin{bmatrix}-0.5\\-2\end{bmatrix} \quad \begin{bmatrix}0.5\\1\end{bmatrix} \quad \begin{bmatrix}1.5\\2\end{bmatrix} \quad \begin{bmatrix}2.5\\3\end{bmatrix}$$

Example#1: PCA

4. Calculate the covariance matrix

Covariance Matrix = $\frac{\Sigma (x_i - \mu)(x_i - \mu)^t}{n}$

$$m_1 = (X_1 - \mu)(X_1 - \mu)^T = \begin{bmatrix} -2.5 \\ -4 \end{bmatrix} [-2.5 \quad -4] = \begin{bmatrix} 6.25 & 10 \\ 10 & 16 \end{bmatrix}$$

$$m_2 = (X_2 - \mu)(X_2 - \mu)^T = \begin{bmatrix} -1.5 \\ 0 \end{bmatrix} [-1.5 \quad 0] = \begin{bmatrix} 2.25 & 0 \\ 0 & 0 \end{bmatrix}$$

$$m_3 = (X_3 - \mu)(X_3 - \mu)^T = \begin{bmatrix} -0.5 \\ -2 \end{bmatrix} [-0.5 \quad -2] = \begin{bmatrix} 0.25 & 1 \\ 1 & 4 \end{bmatrix}$$

$$m_4 = (X_4 - \mu)(X_4 - \mu)^T = \begin{bmatrix} 0.5 \\ 1 \end{bmatrix} [0.5 \quad 1] = \begin{bmatrix} 0.25 & 0.5 \\ 0.5 & 1 \end{bmatrix}$$

$$m_5 = (X_5 - \mu)(X_5 - \mu)^T = \begin{bmatrix} 1.5 \\ 2 \end{bmatrix} [1.5 \quad 2] = \begin{bmatrix} 2.25 & 3 \\ 3 & 4 \end{bmatrix}$$

$$m_6 = (X_6 - \mu)(X_6 - \mu)^T = \begin{bmatrix} 2.5 \\ 3 \end{bmatrix} [2.5 \quad 3] = \begin{bmatrix} 6.25 & 7.5 \\ 7.5 & 9 \end{bmatrix}$$

Covariance matrix

$$= (m_1 + m_2 + m_3 + m_4 + m_5 + m_6) / 6 = \frac{1}{6} \begin{bmatrix} 17.5 & 22 \\ 22 & 34 \end{bmatrix} =$$

$$\begin{bmatrix} 2.92 & 3.67 \\ 3.67 & 5.67 \end{bmatrix}$$

Example#1: PCA

Calculate the eigen values and eigen vectors of the covariance matrix.

λ is an eigen value for a matrix M if it is a solution of the characteristic equation $|M - \lambda I| = 0$.

$$\begin{bmatrix} 2.92 & 3.67 \\ 3.67 & 5.67 \end{bmatrix} - \begin{bmatrix} \lambda & 0 \\ 0 & \lambda \end{bmatrix} = 0 \qquad \begin{bmatrix} 2.92 - \lambda & 3.67 \\ 3.67 & 5.67 - \lambda \end{bmatrix} = 0$$

$(2.92 - \lambda)(5.67 - \lambda) - (3.67 \times 3.67) = 0$

$16.56 - 2.92\lambda - 5.67\lambda + \lambda^2 - 13.47 = 0$

$\lambda = 8.22, 0.38,$

$\lambda_1 = 8.22$ and $\lambda_2 = 0.38$.

Clearly, the second eigen value is very small compared to the first eigen value.

So, the second eigen vector can be left out.

Example#1: PCA

Eigen vector corresponding to the greatest eigen value is the principal component for the given data set.

So. we find the eigen vector corresponding to eigen value λ_1.

We use the following equation to find the eigen vector-

$$MX = \lambda X$$

M = Covariance Matrix, X = Eigen vector, λ = Eigen value

$$\begin{bmatrix} 2.92 & 3.67 \\ 3.67 & 5.67 \end{bmatrix} \begin{bmatrix} X_1 \\ X_2 \end{bmatrix} = 8.22 \begin{bmatrix} X_1 \\ X_2 \end{bmatrix}$$

$2.92X_1 + 3.67X_2 = 8.22X_1$

$3.67X_1 + 5.67X_2 = 8.22X_2$

$\Rightarrow$ $X_1 = 2.55$, $X_2 = 3.67$ $\Rightarrow$ Eigen Vector: $\begin{bmatrix} X_1 \\ X_2 \end{bmatrix} = \begin{bmatrix} 2.55 \\ 3.67 \end{bmatrix}$

Principal component: $\begin{bmatrix} X_1 \\ X_2 \end{bmatrix} = \begin{bmatrix} 2.55 \\ 3.67 \end{bmatrix}$

Example#2: PCA

Use PCA Algorithm to transform the pattern (2, 1) onto the eigen vector in the previous question.

Given Feature Vector: $\begin{bmatrix}2\\1\end{bmatrix}$

The feature vector gets transformed to

= Transpose of Eigen vector x (Feature Vector – Mean Vector)

$$=\begin{bmatrix}2\\1\end{bmatrix}^T\left(\begin{bmatrix}2\\1\end{bmatrix}-\begin{bmatrix}4.5\\5\end{bmatrix}\right)$$

$$=\begin{bmatrix}2.55 & 3.66\end{bmatrix}\begin{bmatrix}-2.5\\-4\end{bmatrix}$$

$$= -21.055$$

Example#3

Large Size Apples	Rotten Apples	Damaged Apples	Small Apples
F1	F2	F3	F4
1	5	3	1
4	2	6	3
1	4	3	2
4	4	1	1
5	5	2	3

	F1	F2	F3	F4
Mean	3	4	3	2
SD	1.87	1.223	1.87	1

Example#3: 1. Standardized Data Set

	F1	F2	F3	F4
Mean	3	4	3	2
SD	1.87	1.223	1.87	1
Standardized Dataset	-1.0695	0.8196	0	-1
	0.5347	-1.6393	1.6042	1
	-1.0695	0	0	0
	0.5347	0	-1.0695	-1
	1.0695	0.8196	-0.5347	1

$$Sample\ SD, S = \sqrt{\frac{\sum_{i=1}^{n}(x_i-\overline{x})^2}{n-1}} = \sqrt{\frac{(1-3)^2+(4-3)^2+(1-3)^2+(4-3)^2+(5-3)^2}{5-1}} = 1.87$$

$$\text{Standardized Data, } Z = \frac{x_i-\mu}{\sigma} = \frac{x_i-\overline{x}}{s} = \frac{1-3}{1.87} = -1.0695$$

Example#3: 2. Covariance Matrix Computation

	F1	F2	F3	F4
F1	Var(F1)	CoV(F1,F2)	CoV(F1,F3)	CoV(F1,F4)
F2	CoV(F2,F1)	Var(F2)	CoV(F2,F3)	CoV(F2,F4)
F3	CoV(F3,F1)	CoV(F3,F2)	Var(F3)	CoV(F3,F4)
F4	CoV(F4,F1)	CoV(F4,F2)	CoV(F4,F3)	Var(F4)

Example#3: 2. Covariance Matrix Computation

	F1	F2	F3	F4
Mean	3	4	3	2
SD	1.87	1.223	1.87	1
Standardized Dataset	-1.0695	0.8196	0	-1
	0.5347	-1.6393	1.6042	1
	-1.0695	0	0	0
	0.5347	0	-1.0695	-1
	1.0695	0.8196	-0.5347	1

Since you have already standardized the features, you can consider Mean = 0 and Standard Deviation=1 for each feature.

VAR(F1, F1) = $((-1.0695-0)^2 + (0.5347-0)^2 + (-1.0695-0)^2 + (0.5347-0)^2 + (1.069-0)^2)/5 = 0.78$

Example#3: 2. Covariance Matrix Computation

	F1	F2	F3	F4
Mean	3	4	3	2
SD	1.87	1.223	1.87	1
Standardized Dataset	-1.0695	0.8196	0	-1
	0.5347	-1.6393	1.6042	1
	-1.0695	0	0	0
	0.5347	0	-1.0695	-1
	1.0695	0.8196	-0.5347	1

COV(F1,F2) = ((-1.0695–0)*(0.8196-0)* + *(0.5347–0)*(-1.6393-0) + (-1.0695–0)* (0.0000-0) + (0.5347–0)*(0.0000-0)+(1.0695–0)* (0.8196–0))/5= -0.8586

Example#3: 2. Covariance Matrix Computation

	F1	F2	F3	F4
F1	0.78	-0.8586	-0.055	0.424
F2	-0.8586	0.78	-0.607	-0.326
F3	-0.055	-0.607	0.78	0.426
F4	0.424	-0.326	0.424	0.78

Example#3: 3. Feature Vector

Let A be any square matrix. A non-zero vector v is an eigenvector of A if $Av = \lambda v$. λ is called eigenvalue associated with eigenvector v of A.

$\det(A - \lambda I) = 0$, you will get the following matrix.

	F1	F2	F3	F4
F1	0.78-λ	-0.8586	-0.055	0.424
F2	-0.8586	0.78-λ	-0.607	-0.326
F3	-0.055	-0.607	0.78-λ	0.426
F4	0.424	-0.326	0.426	0.78-λ

$$\begin{vmatrix} 0.78-\lambda & -0.8586 \\ -0.8586 & 0.78-\lambda \end{vmatrix} = \left|0.6084 - 1.56\lambda + \lambda^2 - 0.7372\right| = 0$$

$$\begin{vmatrix} 0.78-\lambda & 0.426 \\ 0.426 & 0.78-\lambda \end{vmatrix} = \left|0.6084 - 1.56\lambda + \lambda^2 - 0.1815\right| = 0$$

Example#3: 3. Feature Vector

det(A- λI) = 0, you will get the following matrix.

$$\begin{vmatrix} 0.78-\lambda & -0.8586 \\ -0.8586 & 0.78-\lambda \end{vmatrix} = \left|0.6084 - 1.56\lambda + \lambda^2 - 0.7372\right|$$

$$= \left|\lambda^2 - 1.56\lambda - 0.1288\right| = 0$$

$$\lambda = \left|\frac{1.56 \pm \sqrt{2.4336+0.5152}}{2}\right| = \left|\frac{1.56 \pm 1.7172}{2}\right| = 0.0786, 1.6386$$

$$\begin{vmatrix} 0.78-\lambda & 0.426 \\ 0.426 & 0.78-\lambda \end{vmatrix} = \left|0.6084 - 1.56\lambda + \lambda^2 - 0.1815\right| = 0$$

$$= \left|\lambda^2 - 1.56\lambda - 0.4269\right| = 0$$

$$\lambda = \left|\frac{1.56 \pm \sqrt{2.4336+1.7076}}{2}\right| = \left|\frac{1.56 \pm 2.035}{2}\right| = 0.2375, 1.7975$$

$$\lambda = 1.7975, 1.6386, 0.2375, 0.0786$$

Example#3: 4. Feature Vector

Let, $V = \begin{bmatrix} v_1 \\ v_2 \end{bmatrix}$ From, (A- λI)V = 0

$$\begin{bmatrix} 0.78-\lambda & -0.8586 \\ -0.8586 & 0.78-\lambda \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \Rightarrow \begin{bmatrix} (0.78-\lambda)v_1 - 0.8586v_2 \\ -0.8586v_1 + (0.78-\lambda)v_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$(0.78-\lambda)v_1 - 0.8586v_2 = 0$$
$$-0.8586v_1 + (0.78-\lambda)v_2 = 0$$

$$\Rightarrow \quad \frac{v_1}{0.8586} = \frac{v_2}{0.78-\lambda} = t\ (Say)$$

$$v_1 = 0.8586t \qquad v_2 = (0.78-\lambda)t$$

$$t = 1 \Rightarrow V_1 = \begin{bmatrix} 0.8586 \\ 0.78-\lambda \end{bmatrix}$$

λ= 1.7975, 1.6386, 0.2375, 0.0786

$$\|V_1\| = \sqrt{0.8586^2 + (0.78-\lambda)^2} = \sqrt{0.7372 + (0.78-1.7975)^2} = 1.3314$$

Example#3: 4. Feature Vector

$\lambda = 1.7975,\ 1.6386,\ 0.2375,\ 0.0786$

$$e_1 = \begin{bmatrix} \frac{0.8586}{\|V_1\|} \\ \frac{0.78-\lambda}{\|V_1\|} \end{bmatrix} = \begin{bmatrix} \frac{0.8586}{1.3314} \\ \frac{0.78-1.7975}{1.3314} \end{bmatrix} = \begin{bmatrix} 0.6449 \\ -0.7642 \end{bmatrix} \qquad e_2 = \begin{bmatrix} 0.7642 \\ 0.6449 \end{bmatrix}$$

$$\|V_1\| = \sqrt{0.8586^2 + (0.78-\lambda)^2} = \sqrt{0.7372 + (0.78-1.6386)^2} = 1.2142$$

$$e_1 = \begin{bmatrix} \frac{0.8586}{1.2142} \\ \frac{0.78-1.6386}{1.2142} \end{bmatrix} = \begin{bmatrix} 0.7071 \\ -0.7071 \end{bmatrix} \qquad e_2 = \begin{bmatrix} 0.7071 \\ 0.7071 \end{bmatrix}$$

$$\|V_1\| = \sqrt{0.8586^2 + (0.78-\lambda)^2} = \sqrt{0.7372 + (0.78-0.2375)^2} = 1.0156$$

$$e_1 = \begin{bmatrix} \frac{0.8586}{1.0156} \\ \frac{0.78-0.2375}{1.0156} \end{bmatrix} = \begin{bmatrix} 0.8454 \\ 0.5342 \end{bmatrix}$$

$$e_2 = \begin{bmatrix} -0.5342 \\ 0.8454 \end{bmatrix}$$

$$\|V_1\| = \sqrt{0.7372 + (0.78-0.0786)^2} = 1.2292$$

$$e_1 = \begin{bmatrix} \frac{0.8586}{1.2292} \\ \frac{0.78-0.0786}{1.2292} \end{bmatrix} = \begin{bmatrix} 0.6985 \\ 0.5706 \end{bmatrix} \qquad e_2 = \begin{bmatrix} -0.5706 \\ 0.6985 \end{bmatrix}$$

Example#3: 4. Feature Vector

$$e_1 = \begin{bmatrix} 0.6449 \\ -0.7642 \end{bmatrix} \quad e_2 = \begin{bmatrix} 0.7642 \\ 0.6449 \end{bmatrix}$$

$$e_1 = \begin{bmatrix} 0.7071 \\ -0.7071 \end{bmatrix} \quad e_2 = \begin{bmatrix} 0.7071 \\ 0.7071 \end{bmatrix}$$

$$e_1 = \begin{bmatrix} 0.8454 \\ 0.5342 \end{bmatrix} \quad e_2 = \begin{bmatrix} -0.5342 \\ 0.8454 \end{bmatrix}$$

$$e_1 = \begin{bmatrix} 0.6985 \\ 0.5706 \end{bmatrix} \quad e_2 = \begin{bmatrix} -0.5706 \\ 0.6985 \end{bmatrix}$$

E1	E2	E3	E4
0.6449	0.7071	0.8454	0.6985
-0.7642	-0.7071	0.5342	0.5706
0.7642	0.7071	-0.5342	-0.5706
0.6449	0.7071	0.8454	0.6985

Example#3: 4. Feature Vector

E1	E2	E3	E4
0.6449	0.7071	0.8454	0.6985
-0.7642	-0.7071	0.5342	0.5706
0.7642	0.7071	-0.5342	-0.5706
0.6449	0.7071	0.8454	0.6985
1.2898	1.4142	1.6908	1.3970

Sum

E2	E3
0.7071	0.8454
-0.7071	0.5342
0.7071	-0.5342
0.7071	0.8454
1.4142	1.6908

(Top Most) These are your Principal components.

E2	E3
0.7071	0.8454
0.7071	0.8454
0.7071	0.5342
-0.7071	-0.5342
1.4142	1.6908

Example#3: 5. RECAST THE DATA ALONG THE PRINCIPAL COMPONENTS AXES

	F1	F2	F3	F4
Standardized Dataset	-1.0695	0.8196	0	-1
	0.5347	-1.6393	1.6042	1
	-1.0695	0	0	0
	0.5347	0	-1.0695	-1
	1.0695	0.8196	-0.5347	1

E2	E3
0.7071	0.8454
0.7071	0.8454
0.7071	0.5342
-0.7071	-0.5342
1.4142	1.6908

Final Data Set= Standardized Original Data Set * Feature Vector

Rotten Apples	Damaged Apples
0.57953916	0
-1.15914903	-1.35619068
0	0
0	0.5713269
1.15907832	-0.90407076

Your large dataset is now compressed into a small dataset without any loss of data! This is the significance of Principal Component Analysis.

Example#4: PCA

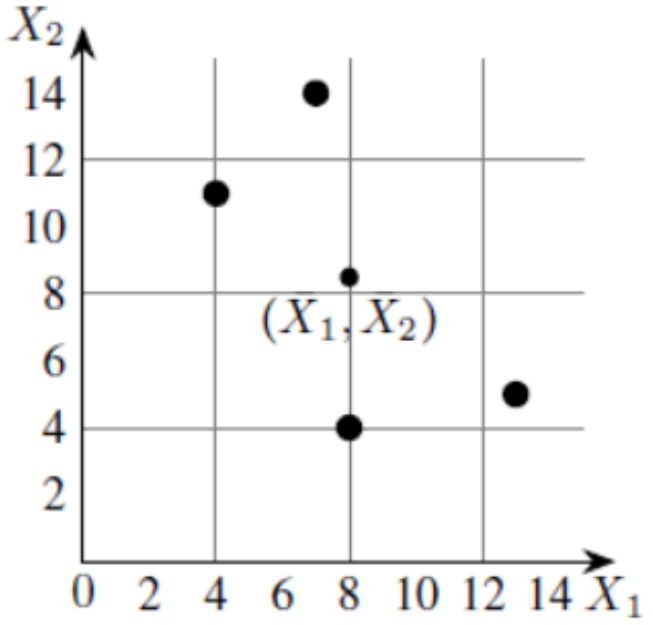
Given the data in Table, reduce the dimension from 2 to 1 using the Principal Component Analysis (PCA) algorithm.

Feature	Example 1	Example 2	Example 3	Example 4
X_1	4	8	13	7
X_2	11	4	5	14

Step 1: Calculate Mean

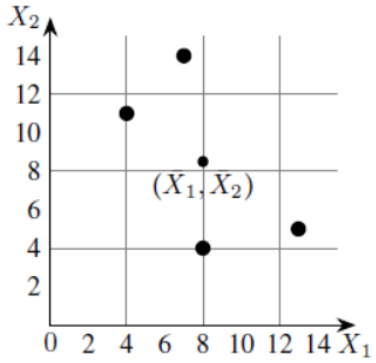
The figure shows the scatter plot of the given data points.

$$\bar{X}_1 = \frac{1}{4}(4 + 8 + 13 + 7) = 8,$$
$$\bar{X}_2 = \frac{1}{4}(11 + 4 + 5 + 14) = 8.5.$$



Example#4: PCA

Step 2: Calculation of the covariance matrix.



The covariances are calculated as follows:

$$\begin{aligned}
\text{Cov}(X_1, X_2) &= \frac{1}{N-1}\sum_{k=1}^{N}(X_{1k} - \bar{X}_1)^2 \\
&= \tfrac{1}{3}\left((4-8)^2 + (8-8)^2 + (13-8)^2 + (7-8)^2\right) \\
&= 14
\end{aligned}$$

$$\begin{aligned}
\text{Cov}(X_1, X_2) &= \frac{1}{N-1}\sum_{k=1}^{N}(X_{1k} - \bar{X}_1)(X_{2k} - \bar{X}_2) \\
&= \tfrac{1}{3}((4-8)(11-8.5) + (8-8)(4-8.5) \\
&\quad + (13-8)(5-8.5) + (7-8)(14-8.5) \\
&= -11
\end{aligned}$$

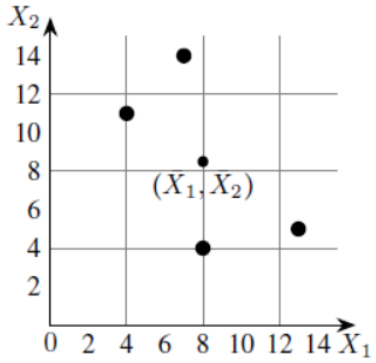
$$\begin{aligned}
\text{Cov}(X_2, X_1) &= \text{Cov}(X_1, X_2) \\
&= -11
\end{aligned}$$

$$\begin{aligned}
\text{Cov}(X_2, X_2) &= \frac{1}{N-1}\sum_{k=1}^{N}(X_{2k} - \bar{X}_2)^2 \\
&= \tfrac{1}{3}\left((11-8.5)^2 + (4-8.5)^2 + (5-8.5)^2 + (14-8.5)^2\right) \\
&= 23
\end{aligned}$$

Example#4: PCA

The covariance matrix is,

$$S = \begin{bmatrix} \text{Cov}(X_1, X_1) & \text{Cov}(X_1, X_2) \\ \text{Cov}(X_2, X_1) & \text{Cov}(X_2, X_2) \end{bmatrix}$$
$$= \begin{bmatrix} 14 & -11 \\ -11 & 23 \end{bmatrix}$$



Step 3: Eigenvalues of the covariance matrix

The characteristic equation of the covariance matrix is,

$$0 = \det(S - \lambda I)$$
$$= \begin{vmatrix} 14 - \lambda & -11 \\ -11 & 23 - \lambda \end{vmatrix}$$
$$= (14 - \lambda)(23 - \lambda) - (-11) \times (-11)$$
$$= \lambda^2 - 37\lambda + 201$$

Solving the characteristic equation we get,

$$\lambda = \tfrac{1}{2}(37 \pm \sqrt{565})$$
$$= 30.3849,\ 6.6151$$
$$= \lambda_1,\ \lambda_2 \quad \text{(say)}$$

Example#4: PCA

Step 4: Computation of the eigenvectors

To find the first principal components, we need only compute the eigenvector corresponding to the largest eigenvalue. In the present example, the largest eigenvalue is λ_1 and so we compute the eigenvector corresponding to λ_1.

The eigenvector corresponding to $\lambda = \lambda_1$ is a vector

$$U = \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}$$

satisfying the following equation:

$$\begin{bmatrix} 0 \\ 0 \end{bmatrix} = (S - \lambda_1 I)X$$
$$= \begin{bmatrix} 14 - \lambda_1 & -11 \\ -11 & 23 - \lambda_1 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}$$
$$= \begin{bmatrix} (14 - \lambda_1)u_1 - 11u_2 \\ -11u_1 + (23 - \lambda_1)u_2 \end{bmatrix}$$

This is equivalent to the following two equations:

$$(14 - \lambda_1)u_1 - 11u_2 = 0$$
$$-11u_1 + (23 - \lambda_1)u_2 = 0$$

Using the theory of systems of linear equations, we note that these equations are not independent and solutions are given by,

Example#4: PCA

$$\frac{u_1}{11} = \frac{u_2}{14 - \lambda_1} = t,$$

that is,

$$u_1 = 11t, \quad u_2 = (14 - \lambda_1)t,$$

where t is any real number.

Taking t = 1, we get an eigenvector corresponding to λ_1 as

$$U_1 = \begin{bmatrix} 11 \\ 14 - \lambda_1 \end{bmatrix}.$$

To find a unit eigenvector, we compute the length of X_1 which is given by,

$$\begin{aligned} \|U_1\| &= \sqrt{11^2 + (14 - \lambda_1)^2} \\ &= \sqrt{11^2 + (14 - 30.3849)^2} \\ &= 19.7348 \end{aligned}$$

Example#4: PCA

Therefore, a unit eigenvector corresponding to λ_1 is

$$e_1 = \begin{bmatrix} 11/\|U_1\| \\ (14-\lambda_1)/\|U_1\| \end{bmatrix}$$
$$= \begin{bmatrix} 11/19.7348 \\ (14-30.3849)/19.7348 \end{bmatrix}$$
$$= \begin{bmatrix} 0.5574 \\ -0.8303 \end{bmatrix}$$

By carrying out similar computations, the unit eigenvector e_2 corresponding to the eigenvalue $\lambda = \lambda_2$ can be shown to be,

$$e_2 = \begin{bmatrix} 0.8303 \\ 0.5574 \end{bmatrix}$$

Example#4: PCA

Step 5: Computation of first principal components

let,

$$\begin{bmatrix} X_{1k} \\ X_{2k} \end{bmatrix}$$

be the k[th] sample in the above Table (dataset). The first principal component of this example is given by (here "T" denotes the transpose of the matrix)

$$e_1^T \begin{bmatrix} X_{1k} - \bar{X}_1 \\ X_{2k} - \bar{X}_2 \end{bmatrix} = \begin{bmatrix} 0.5574 & -0.8303 \end{bmatrix} \begin{bmatrix} X_{1k} - \bar{X}_1 \\ X_{2k} - \bar{X}_2 \end{bmatrix}$$
$$= 0.5574(X_{1k} - \bar{X}_1) - 0.8303(X_{2k} - \bar{X}_2).$$

For example, the first principal component corresponding to the first example

$$\begin{bmatrix} X_{11} \\ X_{21} \end{bmatrix} = \begin{bmatrix} 4 \\ 11 \end{bmatrix}$$

is calculated as follows:

$$\begin{bmatrix} 0.5574 & -0.8303 \end{bmatrix} \begin{bmatrix} X_{11} - \bar{X}_1 \\ X_{21} - \bar{X}_2 \end{bmatrix} = 0.5574(X_{11} - \bar{X}_1) - 0.8303(X_{21} - \bar{X}_2)$$
$$= 0.5574(4 - 8) - 0.8303(11 - 8,5)$$
$$= -4.30535$$

Example#4: PCA

The results of the calculations are summarised in the below Table.

X_1	4	8	13	7
X_2	11	4	5	14
First Principle Components	-4.3052	3.7361	5.6928	-5.1238

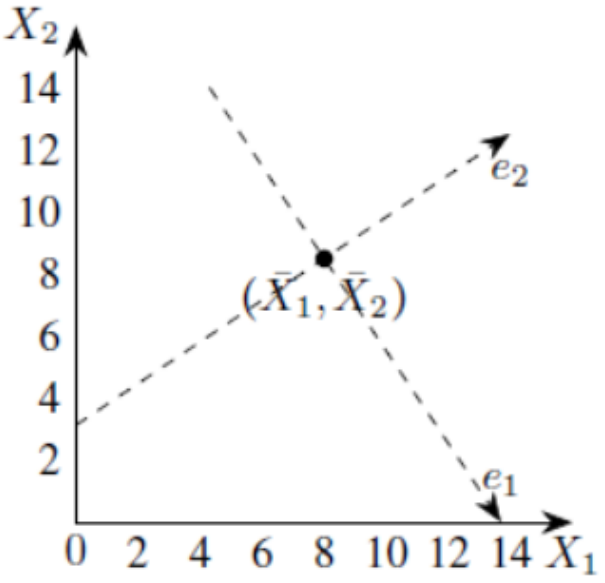
Step 6: Geometrical meaning of first principal components

First, we shift the origin to the "center"

$$(\bar{X}_1, \bar{X}_2)$$

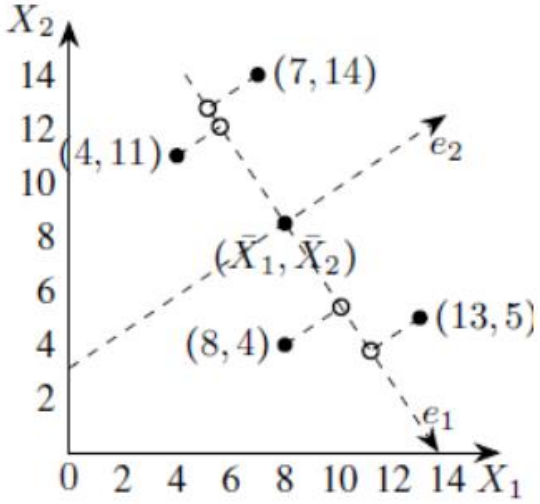
and then change the directions of coordinate axes to the directions of the eigenvectors e_1 and e_2.

Example#4: PCA



The coordinate system for principal components

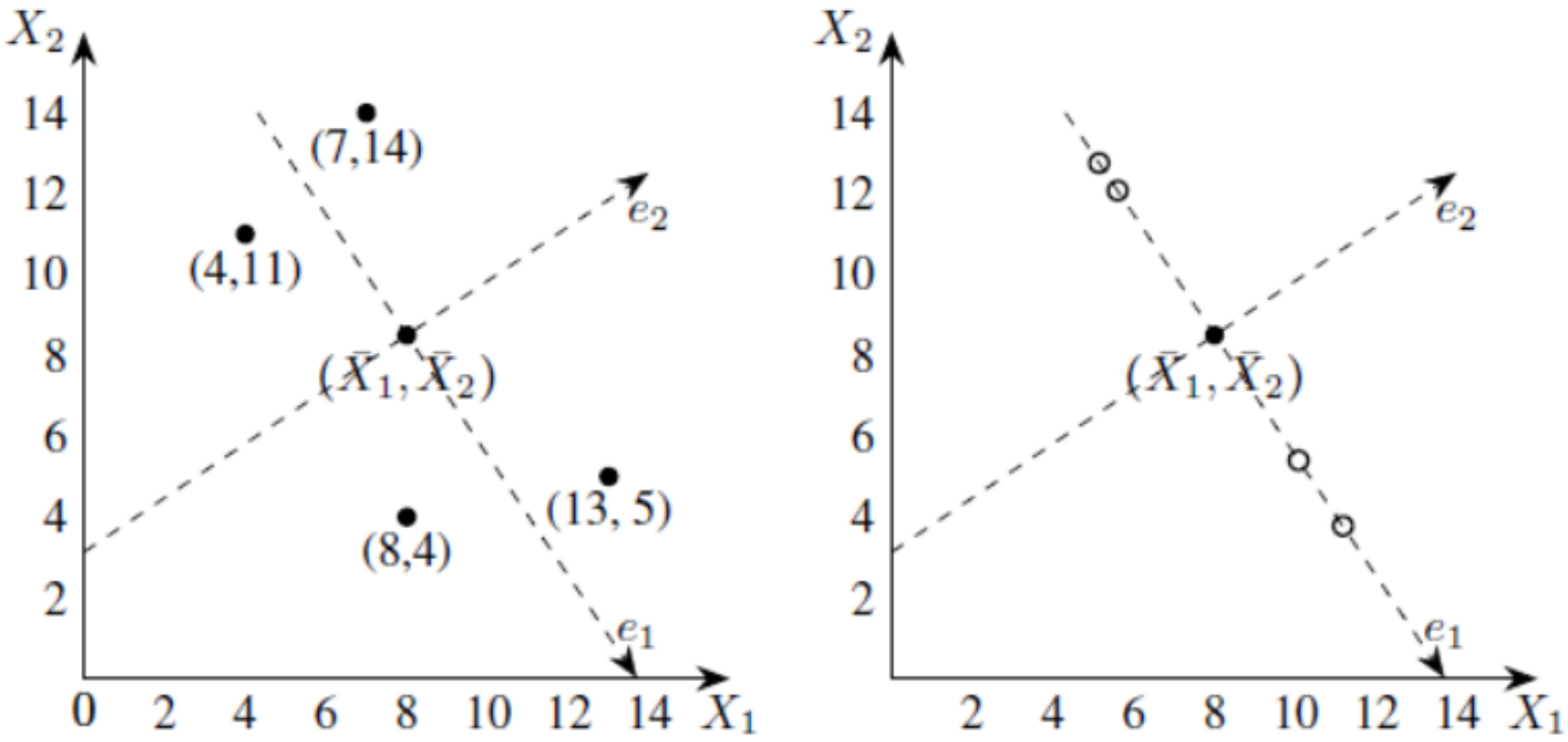
Next, we drop perpendiculars from the given data points to the e1-axis (see below Figure).



Projections of data points on the axis of the first principal component

- The first principal components are the e1-coordinates of the feet of perpendiculars, that is, the projections on the e1-axis.
- The projections of the data points on the e1-axis may be taken as approximations of the given data points hence we may replace the given data set with these points.
- Each of these approx. can be unambiguously specified by a single number, namely, e1-coordinate of approximation.
- Thus the two-dimensional data set can be represented approximately by the following one-dimensional data set.

Example#4: PCA



Geometrical representation of one-dimensional approximation to the data set

Example#5: PCA

Consider the following dataset

x1	2.5	0.5	2.2	1.9	3.1	2.3	2.0	1.0	1.5	1.1
x2	2.4	0.7	2.9	2.2	3.0	2.7	1.6	1.1	1.6	0.9

Step 1: Standardize the Dataset

Mean for x_1 = 1.81 = x_{1mean}

Mean for x_2 = 1.91 = x_{2mean}

We will change the dataset.

$x_{1new} = x_1 - x_{1mean}$	0.69	-1.31	0.39	0.09	1.29	0.49	0.19	-0.81	-0.31	-0.71
$x_{2new} = x_2 - x_{2mean}$	0.49	-1.21	0.99	0.29	1.09	0.79	-0.31	-0.81	-0.31	-1.01

Example#5: PCA

Step 2: Find the Eigenvalues and eigenvectors

$$\textbf{Correlation Matrix c = } C = \left(\frac{X \cdot X^T}{N-1}\right)$$

where, X is the Dataset Matrix (In this numerical, it is a 10 X 2 matrix)

X^T is the transpose of the X (In this numerical, it is a 2 X 10 matrix) and N is the number of elements = 10

So, $C = \left(\frac{X \cdot X^T}{10-1}\right) = \left(\frac{X \cdot X^T}{9}\right)$

{So in order to calculate the Correlation Matrix, we have to do the multiplication of the Dataset Matrix with its transpose}

$$C = \begin{bmatrix} 0.616556 & 0.615444 \\ 0.615444 & 0.716556 \end{bmatrix}$$

Using the equation, **| C – λ I | = 0– equation (i)** where { \lambda is the eigenvalue and I is the Identity Matrix }

Example#5: PCA

So solving equation (i)

$$\begin{bmatrix} 0.616556 & 0.615444 \\ 0.615444 & 0.716556 \end{bmatrix} - \lambda \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$\begin{vmatrix} 0.616556 - \lambda & 0.615444 \\ 0.615444 & 0.716556 - \lambda \end{vmatrix} = 0$$

Taking the determinant of the left side, we get

$$0.44180 - 0.616556\lambda - 0.716556\lambda + \lambda^2 - 0.37877 = 0$$

$$\lambda^2 - 1.33311\lambda + 0.06303 = 0$$

We get two values for λ , that are **(λ_1) = 1.28403 and (λ_2) = 0.0490834**. Now we have to find the eigenvectors for the eigenvalues λ_1 **and** λ_2

Example#5: PCA

To find the eigenvectors from the eigenvalues, we will use the following approach:

First, we will find the eigenvectors for the eigenvalue 1.28403 by using the equation $C \cdot X = \lambda \cdot X$

$$\begin{bmatrix} 0.616556 & 0.615444 \\ 0.615444 & 0.716556 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix} = 1.28403 \cdot \begin{bmatrix} x \\ y \end{bmatrix}$$

$$\begin{bmatrix} 0.616556x + 0.615444y \\ 0.615444x + 0.716556y \end{bmatrix} = \begin{bmatrix} 1.28403x \\ 1.28403y \end{bmatrix}$$

Solving the matrices, we get

0.616556x + 0.615444y = 1.28403x ; **x = 0.922049 y**

(x and y belongs to the matrix X) so if we put y = 1, x comes out to be 0.922049. So now the updated X matrix will look like:

$$X = \begin{bmatrix} 0.922049 \\ 1 \end{bmatrix}$$

Example#5: PCA

IMP: Till now we haven't reached to the eigenvectors, we have to a bit of modifications in the X matrix. They are as follows:

A. Find the square root of the sum of the squares of the element in X matrix i.e.

$$\sqrt{0.922049^2 + 1^2} = \sqrt{0.850174 + 1} = \sqrt{1.850174} = 1.3602$$

B. Now divide the elements of the X matrix by the number 1.3602 (just found that)

$$\begin{bmatrix} \frac{0.922049}{1.3602} \\ \\ \frac{1}{1.3602} \end{bmatrix} = \begin{bmatrix} 0.67787 \\ 0.73518 \end{bmatrix}$$

So now we found the eigenvectors for the eigenvalue λ_1 , they are 0.67787 and 0.73518

Example#5: PCA

Secondly, we will find the eigenvectors for the eigenvalue 0.0490834 by using the equation {Same approach as of previous step)

$$\begin{bmatrix} 0.616556 & 0.615444 \\ 0.615444 & 0.716556 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix} = 0.0490834 \cdot \begin{bmatrix} x \\ y \end{bmatrix}$$

$$\begin{bmatrix} 0.616556x + 0.615444y \\ 0.615444x + 0.716556y \end{bmatrix} = \begin{bmatrix} 0.0490834x \\ 0.0490834y \end{bmatrix}$$

Solving the matrices, we get

0.616556x + 0.615444y = 0.0490834x; **y = -0.922053**

(x and y belongs to the matrix X) so if we put x = 1, y comes out to be -0.922053 So now the updated X matrix will look like:

$$X = \begin{bmatrix} 1 \\ -0.922053 \end{bmatrix}$$

Example#5: PCA

IMP: Till now we haven't reached to the eigenvectors, we have to a bit of modifications in the X matrix. They are as follows:

A. Find the square root of the sum of the squares of the elements in X matrix i.e.

$$\sqrt{1^2 + (-0.922053)^2} = \sqrt{1 + 0.85018} = \sqrt{1.85018} = 1.3602$$

B. Now divide the elements of the X matrix by the number 1.3602 (just found that)

$$\begin{bmatrix} \frac{1}{1.3602} \\ \\ \frac{-0.922053}{1.3602} \end{bmatrix} = \begin{bmatrix} 0.735179 \\ 0.677873 \end{bmatrix}$$

So now we found the eigenvectors for the eigenvector \lambda_2, they are 0.735176 and 0.677873

Sum of eigenvalues (λ_1) and (λ_2) = 1.28403 + 0.0490834 = 1.33 = Total Variance

{Majority of variance comes from λ_1 }

Example#5: PCA

Sum of eigenvalues (λ_1) and (λ_2) = 1.28403 + 0.0490834 = 1.33 = Total Variance

{Majority of variance comes from λ_1}

Step 3: Arrange Eigenvalues

The eigenvector with the highest eigenvalue is the Principal Component of the dataset. So in this case, eigenvectors of lambda1 are the principal components.

{Basically in order to complete the numerical we have to only solve till this step, but if we have to prove why we have chosen that particular eigenvector we have to follow the steps from 4 to 6}

Step 4: Form Feature Vector

$$\begin{bmatrix} 0.677873 & 0.735179 \\ 0.735179 & -0.677879 \end{bmatrix}$$ This is the FEATURE VECTOR for Numerical

Where first column are the eigenvectors of λ_1 & second column are the eigenvectors of λ_2

Example#5: PCA

Step 5: Transform Original Dataset

Use the equation Z = X V

$$\begin{bmatrix} 0.69 & 0.49 \\ -1.31 & -1.21 \\ 0.39 & 0.99 \\ 0.09 & 0.29 \\ 1.29 & 1.09 \\ 0.49 & 0.79 \\ 0.19 & -0.31 \\ -0.81 & -0.81 \\ -0.31 & -0.31 \\ -0.71 & -1.01 \end{bmatrix} \cdot \begin{bmatrix} 0.677873 & 0.735179 \\ 0.735179 & -0.677879 \end{bmatrix} = \begin{bmatrix} 0.8297008 & 0.17511574 \\ -1.77758022 & -0.14285816 \\ 0.99219768 & -0.38437446 \\ 0.27421048 & -0.13041706 \\ 1.67580128 & 0.20949934 \\ 0.91294918 & -0.17528196 \\ -1.14457212 & -0.04641786 \\ -0.43804612 & -0.01776486 \\ -1.22382.62 & 0.16267464 \end{bmatrix} = Z$$

Example#5: PCA

Step 6: Reconstructing Data

Use the equation X = $Z * V^T$ (V^T is Transpose of V), X = Row Zero Mean Data

$$\begin{bmatrix} 0.8297008 & 0.17511574 \\ -1.77758022 & -0.14285816 \\ 0.99219768 & -0.38437446 \\ 0.27421048 & -0.13041706 \\ 1.67580128 & 0.20949934 \\ 0.91294918 & -0.17528196 \\ -1.14457212 & -0.04641786 \\ -0.43804612 & -0.01776486 \\ -1.22382.62 & 0.16267464 \end{bmatrix} \cdot \begin{bmatrix} 0.677873 & 0.735179 \\ 0.735176 & -0.677879 \end{bmatrix} =$$

$$\begin{bmatrix} 0.6899999766573 & 0.4899999834233 \\ -1.3099999556827 & -1.2099999590657 \\ 0.389999968063 & 0.9899999665083 \\ 0.0899999969553 & 0.2899999901893 \\ 0.61212695653593 & 0.35482096313253 \\ 0.4899999834233 & 0.7899999732743 \\ 0.189999935723 & -0.309999995127 \\ -0.8099999725977 & -0.8099999725977 \\ -0.3099999895127 & -0.3099999895127 \\ -0.7099999759807 & -1.0099999658317 \end{bmatrix}$$

Example#5: PCA

So in order to reconstruct the original data, we follow:

Row Original DataSet = Row Zero Mean Data + Original Mean

$$\begin{bmatrix} 0.6899999766573 & 0.4899999834233 \\ -1.3099999556827 & -1.2099999590657 \\ 0.389999968063 & 0.9899999665083 \\ 0.0899999969553 & 0.2899999901893 \\ 0.61212695653593 & 0.35482096313253 \\ 0.4899999834233 & 0.7899999732743 \\ 0.189999935723 & -0.309999995127 \\ -0.8099999725977 & -0.8099999725977 \\ -0.3099999895127 & -0.3099999895127 \\ -0.7099999759807 & -1.0099999658317 \end{bmatrix} + \begin{bmatrix} 1.81 & 1.91 \end{bmatrix} = \begin{bmatrix} 2.49 & 2.39 \\ 0.5 & 0.7 \\ 2.19 & 2.89 \\ 1.89 & 2.19 \\ 3.08 & 2.99 \\ 2.30 & 2.7 \\ 2.01 & 1.59 \\ 1.01 & 1.11 \\ 1.5 & 1.6 \\ 1.1 & 0.9 \end{bmatrix}$$

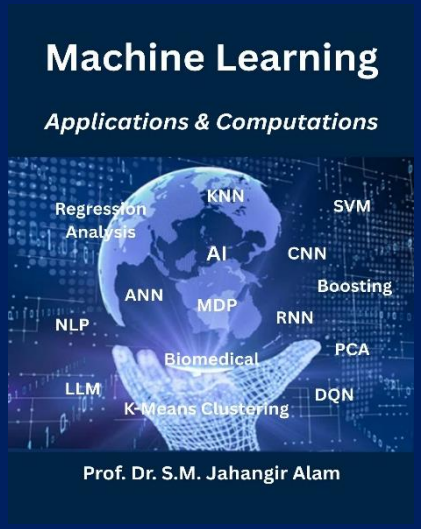
So for the eigenvectors of first eigenvalue, data can be reconstructed similar to the original dataset. Thus we can say that the Principal Component of the dataset is λ_1 is 1.28403 followed by λ_2 **that is 0.0490834**

Thank You

Lecture 3.2

Unsupervised Machine Learning

Independent Component Analysis (ICA)



Instructor Name:

Professor Dr. Jahangir Alam

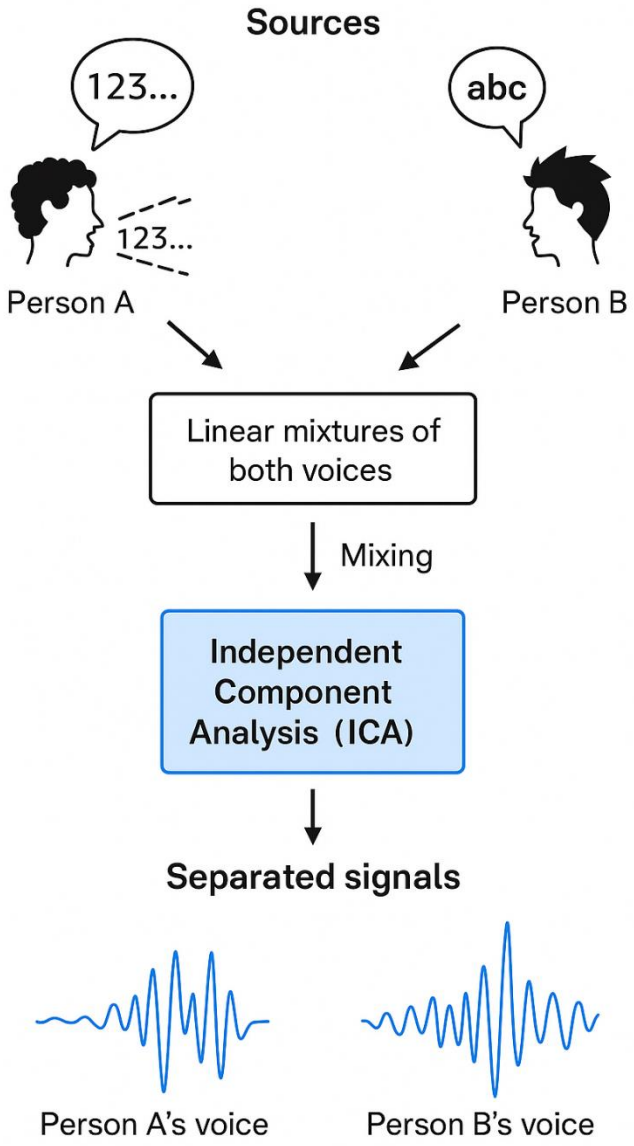
Professor, Dept. of CSE, BAUST

Independent Component Analysis (ICA)

- ✓ ICA is a computational method for separating a multivariate signal into independent non-Gaussian signals.
- ✓ Used in Blind Source Separation (BSS) problems.

- **BSS** is the problem of **recovering original source signals** from a set of **observed mixtures**, **without knowing**:
- The sources, or
- The mixing process.
- That's why it's called **"blind"**.

Independent Component Analysis (ICA)



Independent Component Analysis (ICA)

Observe a mixture of signals and want to recover the original independent sources.

Example:

- Imagine two people talking simultaneously, recorded by two microphones.
- Each microphone records a **linear mixture** of both voices.
- ICA separates them into **independent sources** (the original voices).
- PCA decorrelates data, ICA goes further and makes them **independent**.
- ICA is very powerful for **signal separation** when sources are non-Gaussian.

This is known as the **Cocktail Party Problem.**

Independent Component Analysis (ICA)

Cocktail Party Problem:

- Multiple people are talking at the same time.
- Several microphones record the sounds.
- Each microphone picks up a mixture of voices.
- Task: Recover each person's voice separately.

Applications of ICA:

- **Audio processing** → separating voices/music.
- **Medical signals** → EEG, fMRI signal separation.
- **Image processing** → separating textures, sources.
- **Finance** → independent factors in stock movements.

Independent Component Analysis (ICA)

ICA Model: Steps of ICA Simplified Model,

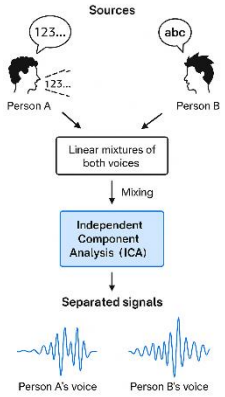
1. Center the data → subtract mean.
2. Whiten the data → transform so covariance matrix = identity.
3. Find independent components by maximizing non-Gaussianity:
   - Uses kurtosis (measure of "tailedness")
   - or negentropy (based on information theory).
4. Find **unmixing matrix** W, Recover sources

   → $s = Wx$. Where, $x = As$

   - x = observed signals (mixtures)
   - A= unknown mixing matrix
   - s = unknown independent source signals

Speakers Speech Detection using ICA

2.4##Speaker's Speech Detection using ICA is a classic application of **Blind Source Separation (BSS)**, where the goal is to separate mixed audio signals (e.g., different speakers recorded by multiple microphones). A simple **2-speaker, 2-microphone**—doing the math manually and explaining intuitively



Problem Setup

We have **2 speakers (sources)**:

- $s_1(t)$: Speaker A saying "Hi"
- $s_2(t)$: Speaker B saying "Bye"

We have **2 microphones** in the room that record **mixed signals** — each mic captures a combination of both speakers. Mathematically, the observed signals (mixtures) are:

$$\begin{bmatrix} x_1(t) \\ x_2(t) \end{bmatrix} = A \begin{bmatrix} s_1(t) \\ s_2(t) \end{bmatrix}$$

where A is the **mixing matrix**.

Speakers Speech Detection using ICA

Step 1: Define Sources and Mixing

Let's choose simple numerical values (samples at a given time):

Mixing matrix:

$$A = \begin{bmatrix} 0.6 & 0.4 \\ 0.3 & 0.7 \end{bmatrix}$$

Compute mixed signals:

$x_1 = 0.6s_1 + 0.4s_2$ and $x_2 = 0.3s_1 + 0.7s_2$

Time	s_1	s_2	x_1	x_2
1	1	2	0.6×1+0.4×2=1.4	0.3×1+0.7×2=1.7
2	2	1	0.6×2+0.4×1=1.6	0.3×2+0.7×1=1.3
3	3	2	0.6×3+0.4×2=2.6	0.3×3+0.7×2=2.3
4	2	3	0.6×2+0.4×3=2.4	0.3×2+0.7×3=2.7

Speakers Speech Detection using ICA

Thus, the **observed signals** are:

$$X = \begin{bmatrix} 1.4 & 1.6 & 2.6 & 2.4 \\ 1.7 & 1.3 & 2.3 & 2.7 \end{bmatrix}$$

Step 2: Centering (Remove Mean)

Compute mean for each microphone:

$$\mu_{x_1} = \frac{1.4 + 1.6 + 2.6 + 2.4}{4} = 2.0$$

$$\mu_{x_2} = \frac{1.7 + 1.3 + 2.3 + 2.7}{4} = 2.0$$

Time	$x_1 - \mu_1$	$x_2 - \mu_2$
1	-0.6	-0.3
2	-0.4	-0.7
3	0.6	0.3
4	0.4	0.7

Centered data matrix:

$$X_c = \begin{bmatrix} -0.6 & -0.4 & 0.6 & 0.4 \\ -0.3 & -0.7 & 0.3 & 0.7 \end{bmatrix}$$

Speakers Speech Detection using ICA

Step 3: Whitening (Remove Correlation)

Compute covariance matrix:

$$C = \frac{1}{4} X_c {X_c}^T$$

$$= \frac{1}{4} \begin{bmatrix} -0.6 & -0.4 & 0.6 & 0.4 \\ -0.3 & -0.7 & 0.3 & 0.7 \end{bmatrix} \begin{bmatrix} -0.6 & -0.4 & 0.6 & 0.4 \\ -0.3 & -0.7 & 0.3 & 0.7 \end{bmatrix}^T$$

$$C = \frac{1}{4} \begin{bmatrix} (-0.6)^2 + (-0.4)^2 + (0.6)^2 + (0.4)^2 & (-0.6)(-0.3) + (-0.4)(-0.7) + (0.6)(0.3) + (0.4)(0.7) \\ (-0.3)(-0.6) + (-0.7)(-0.4) + (0.3)(0.6) + (0.7)(0.4) & (-0.3)2 + (-0.7)2 + (0.3)2 + (0.7)2 \end{bmatrix}$$

$$C_{11} = \frac{0.36 + 0.16 + 0.36 + 0.16}{4} = 0.26$$

$$C_{22} = \frac{0.09 + 0.49 + 0.09 + 0.49}{4} = 0.29$$

$$C = \begin{bmatrix} 0.26 & 0.23 \\ 0.23 & 0.29 \end{bmatrix}$$

$$C_{12} = C_{21} = \frac{0.18 + 0.28 + 0.18 + 0.28}{4} = 0.23$$

Speakers Speech Detection using ICA

- Compute eigenvalues and eigenvectors (by hand, approximately):

$$| C - \lambda I | = 0 \Rightarrow (0.26 - \lambda)(0.29 - \lambda) - (0.23)^2 = 0$$

$$\lambda^2 - 0.55\lambda + (0.0754 - 0.0529) = 0$$

$$\Rightarrow \lambda^2 - 0.55\lambda + 0.0225 = 0$$

$$\lambda_{1,2} = \frac{0.55 \pm \sqrt{0.55^2 - 4(0.0225)}}{2} \Rightarrow \lambda_1 \approx 0.50, \lambda_2 \approx 0.045$$

$$\text{Thus, } D = \begin{bmatrix} 0.50 & 0 \\ 0 & 0.045 \end{bmatrix}$$

Speakers Speech Detection using ICA

Whitening matrix:

$$W_{white} = \sqrt{D^{-1}} = \begin{bmatrix} 1/\sqrt{0.50} & 0 \\ 0 & 1/\sqrt{0.045} \end{bmatrix} = \begin{bmatrix} 1.4142 & 0 \\ 0 & 4.7147 \end{bmatrix}$$

Compute whitened data:

$$Z = W_{white}.X_c = \begin{bmatrix} 1.4142 & 0 \\ 0 & 4.7147 \end{bmatrix}.\begin{bmatrix} -0.6 & -0.4 & 0.6 & 0.4 \\ -0.3 & -0.7 & 0.3 & 0.7 \end{bmatrix}$$

Now Z is uncorrelated.

Why Z is uncorrelated?

Speakers Speech Detection using ICA

Step 4: ICA Iteration (Find Independent Components)

- We find a separating matrix W_{ICA} such that:

$$S_{estimated} = W_{ICA}.Z$$

- ICA maximizes **non-Gaussianity** (e.g., kurtosis) of the components.

- Through iterative algorithms like **FastICA**, we eventually find a matrix W_{ICA} that approximately inverts the mixing A:

$$W_{ICA} \approx A^{-1}$$

Speakers Speech Detection using ICA

Step 5: Recover Speakers

If:

$$A = \begin{bmatrix} 0.6 & 0.4 \\ 0.3 & 0.7 \end{bmatrix} \Rightarrow A^{-1} = \frac{1}{0.42 - 0.12} \begin{bmatrix} 0.7 & -0.4 \\ -0.3 & 0.6 \end{bmatrix}$$
$$= \begin{bmatrix} 2.33 & -1.33 \\ -1.0 & 2.0 \end{bmatrix}$$

So, estimated sources: $\begin{bmatrix} \hat{s}_1(t) \\ \hat{s}_2(t) \end{bmatrix} = A^{-1} \begin{bmatrix} x_1(t) \\ x_2(t) \end{bmatrix}$

$Example\ for\ t = 1$:

$$\begin{bmatrix} \hat{s}_1(t) \\ \hat{s}_2(t) \end{bmatrix} = \begin{bmatrix} 2.33 & -1.33 \\ -1.0 & 2.0 \end{bmatrix} \begin{bmatrix} 1.4 \\ 1.7 \end{bmatrix} = \begin{bmatrix} 2.33(1.4) - 1.33(1.7) \\ -1(1.4) + 2(1.7) \end{bmatrix} = \begin{bmatrix} 1.00 \\ 2.00 \end{bmatrix}$$

That perfectly recovers the **original speakers (s_1=1, s_2=2)**.

Speakers Speech Detection using ICA

Summary:

Step	Operation	Purpose
1	Mix signals X= AS	Simulate microphone recordings
2	Center data	Remove mean
3	Whitening	Remove correlation
4	ICA iteration	Maximize independence
5	Recover signals $S = A^{-1}X$	Separate speaker voices

Speakers Speech Detection using ICA

2.5##Given 3 microphone recordings of a mixture of 3 speakers, use ICA to recover the original signals.

Sample	Speaker 1	Speaker 2	Speaker 3
1	2	0	1
2	1	3	0
3	0	2	4

$$S = \begin{bmatrix} 2 & 0 & 1 \\ 1 & 3 & 0 \\ 0 & 2 & 4 \end{bmatrix}$$

Define a Mixing Matrix

Simulating how microphones capture mixtures:

$$A = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 1 & 1 \\ 2 & 1 & 0 \end{bmatrix}$$

Mix the Sources

$$X = A \cdot S$$

Speakers Speech Detection using ICA

$$A = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 1 & 1 \\ 2 & 1 & 0 \end{bmatrix} \qquad S = \begin{bmatrix} 2 & 0 & 1 \\ 1 & 3 & 0 \\ 0 & 2 & 4 \end{bmatrix}$$

$$X = A \cdot S = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 1 & 1 \\ 2 & 1 & 0 \end{bmatrix} . \begin{bmatrix} 2 & 0 & 1 \\ 1 & 3 & 0 \\ 0 & 2 & 4 \end{bmatrix} = \begin{bmatrix} 4 & 8 & 5 \\ 1 & 5 & 4 \\ 5 & 3 & 2 \end{bmatrix}$$

Speakers Speech Detection using ICA

$$A = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 1 & 1 \\ 2 & 1 & 0 \end{bmatrix} \xrightarrow{\text{Mix the Sources } X = A \cdot S} X = \begin{bmatrix} 4 & 8 & 5 \\ 1 & 5 & 4 \\ 5 & 3 & 2 \end{bmatrix}$$

ICA Conceptually (Recovering the Sources)

In ICA, we compute the unmixing matrix W$\approx A^{-1}$W, and then:

$$S \approx W \cdot X$$

In real ICA:

- We center and whiten the data,
- Then apply an iterative optimization to maximize statistical independence (e.g., FastICA).

But here, for demonstration, we'll manually invert the matrix A and check if we can approximately recover S.

Speakers Speech Detection using ICA

Invert the Mixing Matrix *A*

$$A = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 1 & 1 \\ 2 & 1 & 0 \end{bmatrix} \qquad \mathrm{W}{=}A^{-1} = \begin{bmatrix} -1 & 1 & -1 \\ 2 & -2 & 1 \\ -2 & 1 & 1 \end{bmatrix}$$

$$s_{ij} = \sum_{k=1}^{3} W_{ik} X_{kj}$$

Multiply to Recover,

$$S = W.X = \begin{bmatrix} -1 & 1 & -1 \\ 2 & -2 & 1 \\ -2 & 1 & 1 \end{bmatrix} . \begin{bmatrix} 4 & 8 & 5 \\ 1 & 5 & 4 \\ 5 & 3 & 2 \end{bmatrix} = \begin{bmatrix} -5 & -6 & -3 \\ 6 & 9 & 4 \\ -1 & -8 & -4 \end{bmatrix}$$

Speakers Speech Detection using ICA

Resulting Recovered *S* (Approximation of Sources):

This matrix has the same structural pattern as the original *S* (scaled and sign-flipped — common in ICA):

- ICA can only recover the signals up to permutation and scaling.

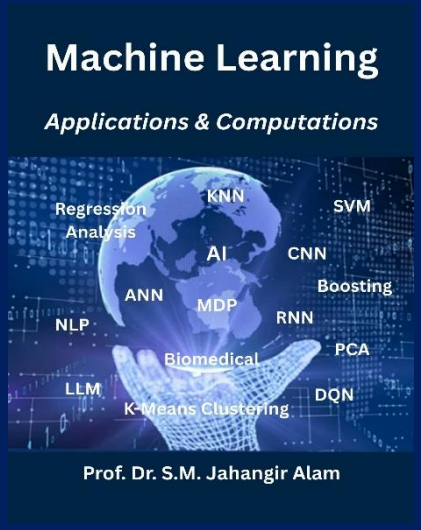
- That means it can't tell which signal belongs to whom, but it can separate them.

Thank You

Lecture 3.3

Unsupervised Machine Learning

K-means Clustering Algorithm



Instructor Name:

Professor Dr. Jahangir Alam

Professor, Dept. of CSE, BAUST

K-means Clustering Algorithm

- An unsupervised ML, partition clustering algorithm used to group a dataset into k clusters.
- Iterative algorithm by selecting randomly k centroids or $\sqrt{k}$ in the dataset.
- After selecting the centroids, the entire dataset is divided into clusters based on the distance of the data points from the centroid.
- In new clusters, the centroids are calculated by taking mean of data points.
- With the new centroids, we regroup the dataset into new clusters.
- Continues until we get a stable cluster.
- We call partition clustering because of k-means clustering algorithm partitions the entire dataset into mutually exclusive clusters.

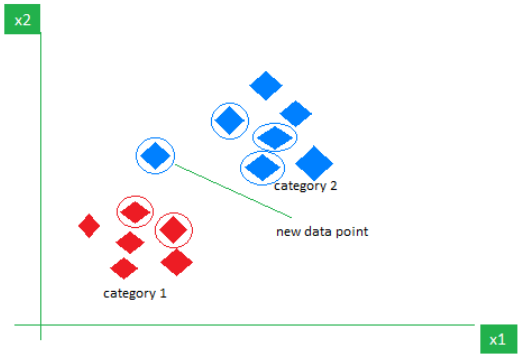
K-means Clustering Algorithm

- A dataset of N entries and a number K as the number of clusters, then steps to find the clusters using the k-means algorithm.

1. select K random entries from dataset and use them as centroids.
2. Find the distance from data point to centroids. (Use any distance metric Euclidean distance, Manhattan distance, or squared Euclidean distance).
3. Start assigning the data points to clusters. Assign each data point to cluster with centroid to the least distance.
4. Calculate the new centroid of the clusters. Using the mean of each data point in the same cluster will be new centroid. If the newly created centroids are the same as the centroids in the previous iteration, consider the current clusters to be final. stop the execution of the algorithm. If any of the newly created centroids is different from the centroids in the previous iteration, go to step 2.

K-means Clustering Algorithm

The new point is classified as Category 2 because most of its closest neighbors are blue squares. KNN assigns the category based on majority of nearby points. The image shows how KNN predicts category of a new data point based on its closest neighbours.



- The red diamonds represent Category 1 and blue squares represent Category 2.

- The new data point checks its closest neighbors (circled points).

- The majority of its closest neighbors are blue squares (Category 2) KNN predicts new data point belongs to Category 2.

Voting for Classification or Taking Average for Regression

- When you want to classify a data point into a category like spam or not spam, the KNN algorithm looks at the K closest points in the dataset. These closest points are called neighbors. The algorithm then looks at which category the neighbors belong to and picks the one that appears the most. This is called majority voting.

- In regression, the algorithm still looks for the K closest points. But instead of voting for a class in classification, it takes the average of the values of those K neighbors. This average is the predicted value for the new point for the algorithm.

What is 'K' in K Nearest Neighbour?

In the k-Nearest Neighbours algorithm k is just a number that tells the algorithm how many nearby points or neighbors to look at when it makes a decision.

Example: Imagine you're deciding which fruit it is based on its shape and size. You compare it to fruits you already know.

- If k = 3, the algorithm looks at the 3 closest fruits to the new one.
- If 2 of those 3 fruits are apples and 1 is a banana, the algorithm says the new fruit is an apple because most of its neighbors are apples.

Choose k for KNN Algorithm?

- The value of k in KNN decides how many neighbors the algorithm looks at when making a prediction.
- Choosing the right k is important for good results.
- If the data has lots of noise or outliers, using a larger k can make the predictions more stable.
- But if k is too large the model may become too simple and miss important patterns and this is called underfitting.
- So k should be picked carefully based on the data.

Statistical Methods for Selecting k

- **Cross-Validation**: Cross-Validation is a good way to find the best value of k is by using k-fold cross-validation.
- This means dividing the dataset into k parts. The model is trained on some of these parts and tested on the remaining ones.
- This process is repeated for each part.
- The k value that gives the highest average accuracy during these tests is usually the best one to use.

Statistical Methods for Selecting k

- **Elbow Method**: In Elbow Method we draw a graph showing the error rate or accuracy for different k values.
- As k increases the error usually drops at first. But after a certain point error stops decreasing quickly.
- The point where the curve changes direction and looks like an "elbow" is usually the best choice for k.
- **Odd Values for k**: It’s a good idea to use an odd number for k especially in classification problems.
- This helps avoid ties when deciding which class is the most common among the neighbors.

K-means Clustering Algorithm

Working of K-Nearest Neighbour

KNN algorithm stores all available cases and classifies new data based on the majority class of its nearest neighbors. Value of **K** in KNN refers to the number of nearest neighbors to consider when performing classification.

K parameter is critical because:

- If **K** is too small, the model may be sensitive to noise in the dataset.
- If **K** is too large, the classification might be too generalized, and nuances in the data may be overlooked.

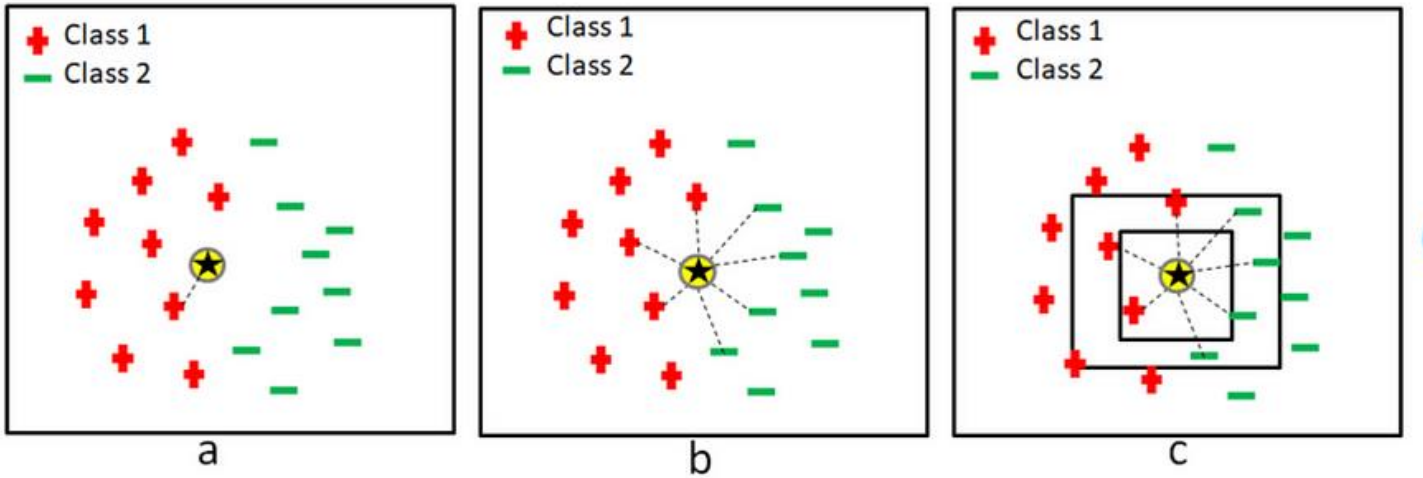
K-means Clustering Algorithm

How do we choose K?

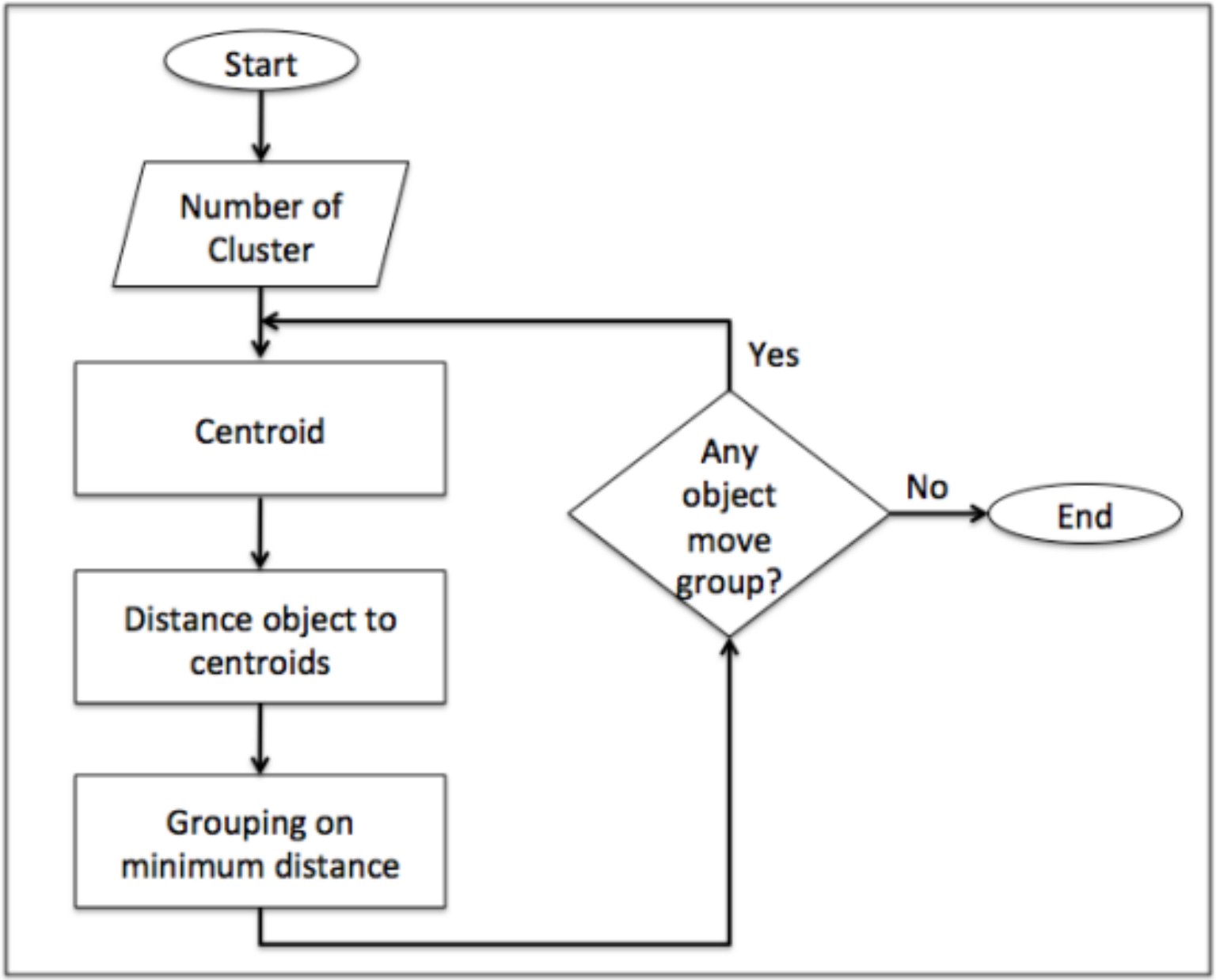
Choosing the right value for **K** is crucial:

- A commonly used rule of thumb is to select $K \approx \sqrt{n}$, where **n** is the number of data points in the dataset.

- If **n** is even adjust **K** to be odd by adding or subtracting 1 to avoid ties in majority voting.



K-means Clustering Algorithm



K-means Clustering Algorithm

$$\text{Euclidean distance} = \sqrt{\sum_{i=1}^{n}(x_i - y_i)^2}$$

$$\text{Class}(x) = \text{mode}(\{y_i | i \in \text{K nearest neighbors of } x\})$$

$$\text{Prediction}(x) = \frac{1}{K}\sum_{i=1}^{K} y_i$$

K-means Clustering Algorithm

Clustering is an unsupervised learning technique that groups data points into clusters such that:

- Points within the same cluster are similar
- Points from different clusters are dissimilar

Unlike classification, no labels are provided in clustering.

Clustering is a process of grouping similar data points together based on their features:

- Points in the same cluster → are similar to each other
- Points in different clusters → are dissimilar

No labels are provided — clustering *discovers hidden patterns* automatically.

K-means Clustering Algorithm

Type	Example Algorithms	Description
1. Partition-based	K-Means, K-Medoids	Divide data into K clusters directly
2. Hierarchical	Agglomerative, Divisive	Build tree (dendrogram) of nested clusters
3. Density-based	DBSCAN, OPTICS	Clusters are dense regions separated by sparse areas
4. Model-based	Gaussian Mixture Models (GMM)	Assume data follows statistical distributions
5. Fuzzy-based	Fuzzy C-Means (FCM)	Soft clustering — partial membership in clusters
6. Grid-based	STING, CLIQUE	Data space divided into grids and clustered

K-means Clustering Algorithm

Clustering Process

Step	Description
1. Input Data	Collect dataset, $X = \{x_1, x_2, \ldots, x_n\}, x_i \in R^d$
2. Choose Algorithm	K-Means, Hierarchical, DBSCAN, Fuzzy C-Means, etc.
3. Define Similarity Measure	Usually, Euclidean distance $\| x_i - \mu_j \|$, but can vary
4. Cluster Assignment	Group points into clusters
5. Evaluate	Check quality using silhouette, Dunn index, etc.

K-means Clustering Algorithm

Applications of Clustering

Domain	Application
Image Processing	Image segmentation, color quantization
Medical Imaging	Tumor detection (MRI, CT, USG)
Marketing	Customer segmentation
Finance	Fraud detection, risk grouping
Text Mining	Topic discovery, document grouping
Remote Sensing	Land-cover classification
Social Networks	Community detection
IoT / Energy	Load pattern clustering

K-means Clustering Algorithm

Solve a numerical problem on k means clustering. The problem is as follows. You are given 15 points in the Cartesian coordinate system as follows.

Point	Coordinates
A1	(2, 10)
A2	(2, 6)
A3	(11, 11)
A4	(6,9)
A5	(6,4)
A6	(1,2)
A7	(5,10)
A8	(4,9)
A9	(10,12)
A10	(7,5)
A11	(9,11)
A12	(4,6)
A13	(3,10)
A14	(3,8)
A15	(6,11)

- make 3 clusters, thus $K \approx \sqrt{15} \approx 3$.
- solve this numerical on k-means clustering.
- First, randomly choose 3 centroids from the given data.
- Let, A2(2,6), A7(5,10), and A15(6,11) as the centroids of the initial clusters. Hence, consider that,
  - Centroid 1=A2(2,6) is associated with cluster 1.
  - Centroid 2=A7(5,10) is associated with cluster 2.
  - Centroid 3=A15(6,11) is associated with cluster 3.

K-means Clustering Algorithm

Point	Coordinates
A1	(2, 10)
A2	(2, 6)
A3	(11, 11)
A4	(6,9)
A5	(6,4)
A6	(1,2)
A7	(5,10)
A8	(4,9)
A9	(10,12)
A10	(7,5)
A11	(9,11)
A12	(4,6)
A13	(3,10)
A14	(3,8)
A15	(6,11)

Choose Number of Clusters, $K \approx \sqrt{15} \approx 3$

- Results from 1st iteration of K means clustering
- Find Euclidean distance between each point and the centroids.
- Minimum distance of each point from centroids
- Assign points to a cluster.

Assign Points to Closest Centroid:

$$Euclidean\ distance, d = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

$$= \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

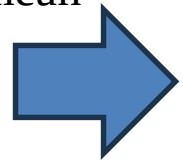
K-means Clustering Algorithm

Point	Distance from centroid 1(2,6)	Distance from centroid 2(5,10)	Distance from centroid 3(6,11)	Assigned Cluster
A1(2, 10)	4	3	4.1231	Cluster 2
A2(2, 6)	0	5	6.4031	Cluster 1
A3(11, 11)	10.2956	6.0827	5	Cluster 3
A4(6,9)	5	1.4142	2	Cluster 2
A5(6,4)	4.4721	6.08276	7	Cluster 1
A6(1,2)	4.1231	8.9442	10.2956	Cluster 1
A7(5,10)	5	0	1.4142	Cluster 2
A8(4,9)	3.6055	1.4142	2.8284	Cluster 2
A9(10,12)	10	5.3851	4.1231	Cluster 3
A10(7,5)	5.0990	5.3851	6.08276	Cluster 1
A11(9,11)	8.6023	4.1231	3	Cluster 3
A12(4,6)	2	4.1231	5.3851	Cluster 1
A13(3,10)	4.1231	2	3.1622	Cluster 2
A14(3,8)	2.2360	2.8284	4.2426	Cluster 1
A15(6,11)	6.4031	1.4142	0	Cluster 3

K-means Clustering Algorithm

Point	Assigned Cluster
A1(2, 10)	Cluster 2
A2(2, 6)	Cluster 1
A3(11, 11)	Cluster 3
A4(6,9)	Cluster 2
A5(6,4)	Cluster 1
A6(1,2)	Cluster 1
A7(5,10)	Cluster 2
A8(4,9)	Cluster 2
A9(10,12)	Cluster 3
A10(7,5)	Cluster 1
A11(9,11)	Cluster 3
A12(4,6)	Cluster 1
A13(3,10)	Cluster 2
A14(3,8)	Cluster 1
A15(6,11)	Cluster 3

Results from 1st iteration of K means clustering, the new centroid mean



New Cluster	Point	New Mean
Cluster 1	A2(2, 6)	(3.833, 5.167)
	A5(6,4)	
	A6(1,2)	
	A10(7,5)	
	A12(4,6)	
	A14(3,8)	
Cluster 2	A1(2, 10)	(4, 9.6)
	A4(6,9)	
	A7(5,10)	
	A8(4,9)	
	A13(3,10)	
Cluster 3	A3(11, 11)	(9, 11.25)
	A9(10,12)	
	A11(9,11)	
	A15(6,11)	

K-means Clustering Algorithm

Point	Distance from New centroid 1 (3.833, 5.167)	Distance from New centroid 2 (4, 9.6)	Distance from New centroid 3 (9,11.25)	New Cluster
A1(2, 10)	5.169	2.040	7.111	Cluster 2
A2(2, 6)	2.013	4.118	8.750	Cluster 1
A3(11, 11)	9.241	7.139	2.016	Cluster 3
A4(6,9)	4.403	2.088	3.750	Cluster 2
A5(6,4)	2.461	5.946	7.846	Cluster 1
A6(1,2)	4.249	8.171	12.230	Cluster 1
A7(5,10)	4.972	1.077	4.191	Cluster 2
A8(4,9)	3.837	0.600	5.483	Cluster 2
A9(10,12)	9.204	6.462	1.250	Cluster 3
A10(7,5)	3.171	5.492	6.562	Cluster 1
A11(9,11)	7.792	5.192	0.250	Cluster 3
A12(4,6)	0.850	3.600	7.250	Cluster 1
A13(3,10)	4.904	1.077	6.129	Cluster 2
A14(3,8)	2.953	1.887	6.824	Cluster 2
A15(6,11)	6.223	2.441	3.010	Cluster 2

K-means Clustering Algorithm

Point	New Cluster
A1(2, 10)	Cluster 2
A2(2, 6)	Cluster 1
A3(11, 11)	Cluster 3
A4(6,9)	Cluster 2
A5(6,4)	Cluster 1
A6(1,2)	Cluster 1
A7(5,10)	Cluster 2
A8(4,9)	Cluster 2
A9(10,12)	Cluster 3
A10(7,5)	Cluster 1
A11(9,11)	Cluster 3
A12(4,6)	Cluster 1
A13(3,10)	Cluster 2
A14(3,8)	Cluster 2
A15(6,11)	Cluster 2

Results from 2nd iteration of K means clustering:

- **New Cluster 1:**

  A2(2,6), A5(6,4), A6(1,2), A10(7,5), A12(4,6)

- **New Cluster 2:**

  A1(2,10), A4(6,9), A7(5,10), A8(4,9), A13(3,10), A14(3,8), A15(6,11)

- **New Cluster 3:**

  A3(11,11), A9(10,12), A11(9,11)

Results from 2nd iteration of K means clustering, the new centroid mean

New Cluster 1: (4, 4.6)

New Cluster 2: (4.143, 9.571)

New Cluster 3: (10,11.333)

K-means Clustering Algorithm

Point	Distance from New centroid 1 (4, 4.6)	Distance from New centroid 2 (4.143, 9.571)	Distance from New centroid 3 (10,11.333)	New Cluster
A1(2, 10)	5.758	2.186	8.110	Cluster 2
A2(2, 6)	2.441	4.165	9.615	Cluster 1
A3(11, 11)	9.485	7.004	1.054	Cluster 3
A4(6,9)	4.833	1.943	4.631	Cluster 2
A5(6,4)	2.088	5.872	8.535	Cluster 1
A6(1,2)	**3.970**	**8.197**	**12.966**	**Cluster 1**
A7(5,10)	5.492	0.958	5.175	Cluster 2
A8(4,9)	4.400	0.589	6.438	Cluster 2
A9(10,12)	9.527	6.341	0.667	Cluster 3
A10(7,5)	3.027	5.390	7.008	Cluster 1
A11(9,11)	**8.122**	**5.063**	**1.054**	**Cluster 3**
A12(4,6)	1.400	3.574	8.028	Cluster 1
A13(3,10)	5.492	1.221	7.126	Cluster 2
A14(3,8)	3.544	1.943	7.753	Cluster 2
A15(6,11)	6.705	2.343	4.014	Cluster 2

K-means Clustering Algorithm

Point	New Cluster
A1(2, 10)	Cluster 2
A2(2, 6)	Cluster 1
A3(11, 11)	Cluster 3
A4(6,9)	Cluster 2
A5(6,4)	Cluster 1
A6(1,2)	Cluster 1
A7(5,10)	Cluster 2
A8(4,9)	Cluster 2
A9(10,12)	Cluster 3
A10(7,5)	Cluster 1
A11(9,11)	Cluster 3
A12(4,6)	Cluster 1
A13(3,10)	Cluster 2
A14(3,8)	Cluster 2
A15(6,11)	Cluster 2

Results from 3rd iteration of K means clustering

- **Next New Cluster 1:**

  A2(2,6), A5(6,4), A6(1,2), A10(7,5), A12(4,6)

- **Next New Cluster 2:**

  A1(2,10), A4(6,9), A7(5,10), A8(4,9), A13(3,10), A14(3,8), A15(6,11)

- **Next New Cluster 3:**

  A3(11,11), A9(10,12), A11(9,11)

Results from 3rd iteration of K means clustering, the new centroid mean

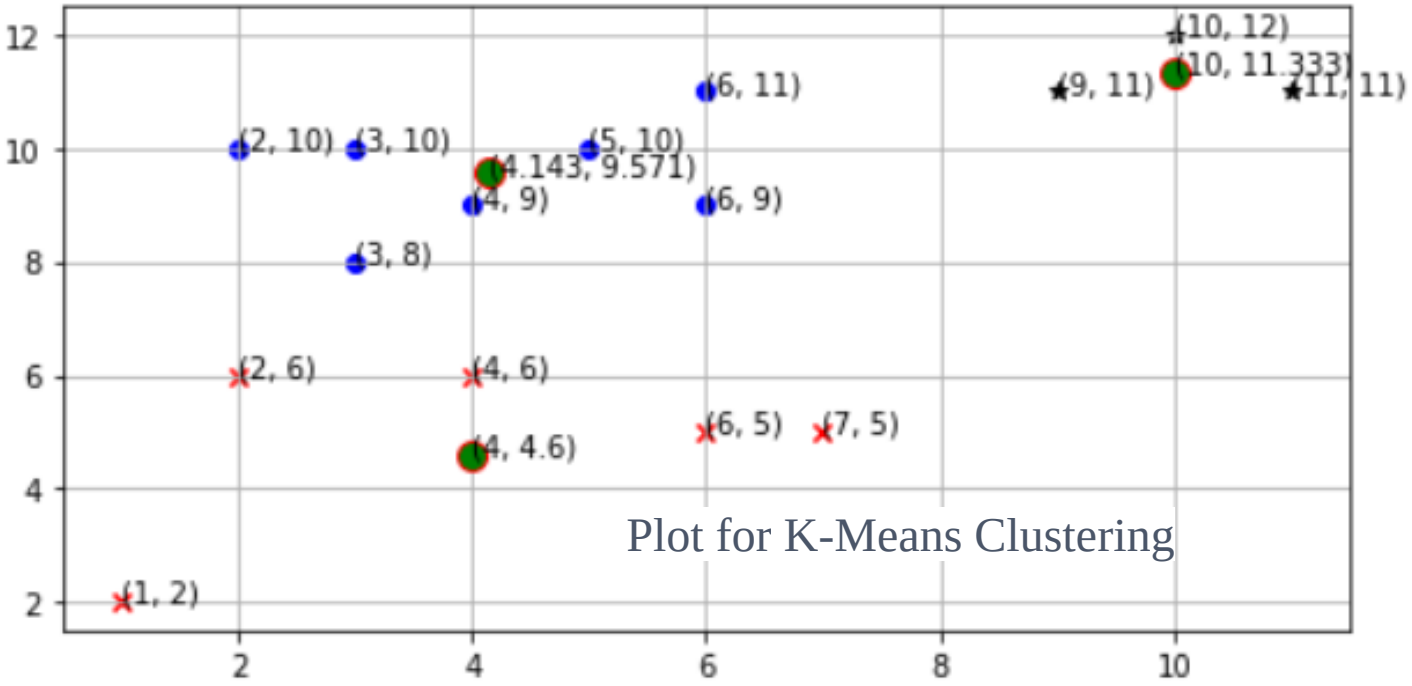
Next New Cluster 1: (4, 4.6)

Next New Cluster 2: (4.143, 9.571)

Next New Cluster 1: (10,11.333)

K-means Clustering Algorithm

- ✓ No point has changed its cluster compared to 2nd and 3rd iteration K-means.
- ✓ Centroid also remains constant.
- ✓ Therefore, we will say that the clusters have been stabilized.
- ✓ Hence, the clusters obtained after the 3rd iteration are the final clusters made from the given dataset.
- ✓ we plot the clusters on a graph.



Plot for K-Means Clustering

Applications of K-means Clustering

- **Document Classification**: divide documents into various clusters based on their content, topics, and tags.

- **Customer segmentation**: Supermarkets and e-commerce websites divide their customers into various clusters based on their transaction data and demography. This helps the business to target appropriate customers with relevant products to increase sales.

- **Cyber profiling**: collect data from individuals as well as groups to identify their relationships. With k-means clustering, easily make clusters of people based on their connection to each other to identify any available patterns.

Applications of K-means Clustering

- **Image segmentation**: to perform image segmentation by grouping similar pixels into clusters.

- **Fraud detection in banking and insurance**: By using historical data on frauds, banks and insurance agencies can predict potential frauds by the application of k-means clustering.

- Ride-share data analysis, social media profiling, identification of crime localities, etc.

Applications of K-means Clustering

✓ **Image Compression by K-means Clustering**

- used for image compression, used to reduce the number of colors in an image while maintaining its visual quality.

- used to cluster the colors in the image and replace the pixels with the centroid color of each cluster, resulting in a compressed image.

✓K-Means clustering is used in a variety of examples or business cases in real life, like:

- Academic performance
- Diagnostic systems
- Search engines
- Wireless sensor networks

Applications of K-means Clustering

Based on scores, students are categorized into grades like A, B, or C.

Diagnostic systems

Smarter medical decision support systems, especially in the treatment of liver ailments.

Search engines

Backbone of search engines. The search results need to be grouped, and the search engines use clustering to do this.

Wireless sensor networks

Finding the cluster heads, which collect all the data in its respective cluster.

Advantages

- **Easy to implement**: Iterable algorithm and a relatively simple algorithm. perform manually with numerical example.
- **Scalability**: use for even 10 records or even 10 million records in a dataset.
- **Convergence**: guaranteed to give convergence results. The result of the execution of the algorithm for ensure.
- **Generalization**: doesn't apply to a specific problem. From numerical data to text documents, any dataset to perform clustering. applied to datasets of different sizes having entirely different distributions in dataset. Thus, this algorithm is completely generalized.
- **Choice of centroids**: choice of centroids in an easy manner. Hence, the algorithm allows to choose and assign centroids that fit well with the dataset.

Disadvantages of K-means Clustering

- **Deciding the number of clusters**: need to decide the number of clusters by using the elbow method.
- **Choice of initial centroids**: The number of iterations completely depends on the choice of centroids. Hence, need to properly choose centroids in initial step for maximizing efficiency of the algorithm.
- **Effect of outliers**: use all the points in a cluster to determine the centroids for the next iteration. If there are outliers in the dataset, they highly affect the position of the centroids. Due to this, the clustering becomes inaccurate. To avoid this, you can try to identify outliers and remove them in the data cleaning process.

Disadvantages of K-means Clustering

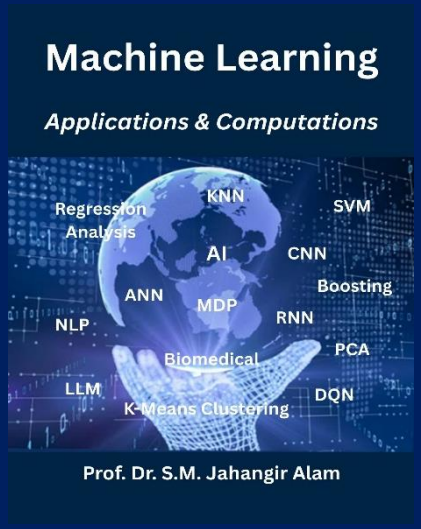
•**Curse of Dimensionality**: If the number of dimensions in the dataset increases, the distance of the data points from a given point starts converging to a specific value. Thus, calculates the clusters based on the distance between the points becomes inefficient. To overcome this problem, use advanced clustering algorithms like spectral clustering. Alternatively, you can also try to reduce the dimensionality of the dataset while data preprocessing.

Thank You

Lecture 3.4

Unsupervised Machine Learning

Fuzzy C-Means (FCM) Clustering



Instructor Name:

Professor Dr. Jahangir Alam

Professor, Dept. of CSE, BAUST

Problem: Predict if a person will default on a bank loan

- A bank wants to assess loan default risk based on:
- Income
- Loan Amount
- Credit Score
- We will use logistic regression, a common supervised ML model for binary classification.

Person	Income ($k)	Loan ($k)	Credit Score	Default (1=yes, 0=no)
A	60	30	700	0
B	40	50	600	1
C	80	20	750	0
D	30	60	580	1

Problem: Predict if a person will default on a bank loan

Logistic Regression Setup

- Let's use a simple linear model to predict probability of default as Linear combination (logit input):

$$z = w_1 \times Income + w_2 \times Loan + w_3 \times CreditScore + b$$

Sigmoid function (output probability):

$$p = \frac{1}{1+e^{-z}} \text{ (probability of default)}$$

We'll hand-calculate using simple weights for intuition:

- $w_1 = -0.02$ (higher income → less likely to default)
- $w_2 = +0.04$ (higher loan → more likely to default)
- $w_3 = -0.005$ (higher credit score → less likely to default)
- $b = -2$

Problem: Predict if a person will default on a bank loan

Predict for a New Person (E): Income = $50k, Loan = $40k, Credit Score = 630

$$z = (-0.02)(50) + (0.04)(40) + (-0.005)(630) + (-2) = -4.55$$

$$p = \frac{1}{1 + e^{-z}} = \frac{1}{1 + e^{4.55}} \approx 0.0105$$

- Probability of default = 1.05% → Very low risk
- Predict: **No default (0)**

Fuzzy C-Means (FCM) Clustering

- **Fuzzy C-Means (FCM)** is a **soft clustering** algorithm used to group data points into clusters, where each data point can belong to **more than one cluster** with varying degrees of membership.
- It's an extension of **K-Means**, but instead of assigning each data point strictly to one cluster, FCM assigns **membership probabilities** (values between 0 and 1).
- In **K-Means**, a data point belongs to exactly one cluster. In **Fuzzy C-Means**, a data point can belong **partially** to multiple clusters.
- Each data point x_i has a membership value u_{ij} indicating the degree of belonging to cluster j.

Fuzzy C-Means (FCM) Clustering

- Unlike K-means, where each point belongs **strictly** to one cluster, FCM assigns **membership values** between 0 and 1 to each point for each cluster.
- Membership values sum to 1 for each point.
- The algorithm minimizes weighted within-cluster sum of squares using membership degrees.

Fuzzy C-Means (FCM) Clustering

- In real-world data, **boundaries between clusters are not always sharp.**
- FCM captures this uncertainty by giving **membership values** (between 0 and 1).
- The **sum of memberships for each data point = 1.**

Fuzziness Parameter m

- $m = 1$: reduces to **hard clustering** (K-Means)
- $m > 1$: gives **softer memberships** (usually $m = 2$)

Key Difference vs K-Means

Feature	K-Means	Fuzzy C-Means
Membership	Hard (0 or 1)	Soft (0–1): $0 \leq u_{ij} \leq 1$
Objective	Minimize total variance	Minimize weighted variance (by membership)
Sensitivity	Sensitive to noise	Less sensitive due to fuzzy weights
Output	Single cluster per point	Membership matrix
Use case	Clear boundaries	Overlapping classes (e.g., medical images)

Applications of Fuzzy C-Means

Domain	Application
Medical Imaging	Brain tumor, tissue segmentation in MRI/CT/Ultrasound
Remote Sensing	Land cover classification (forest, water, urban)
Pattern Recognition	Handwriting, texture, and face recognition
Image Compression	Cluster color/intensity values
Data Analysis	Overlapping cluster identification
Finance	Credit risk and customer segmentation
Bioinformatics	Gene expression clustering

FCM Algorithm Steps

1. Initialize:

- Choose number of clusters C.
- Initialize membership matrix $U = [u_{ij}]$ randomly (each row sums to 1)
- Let, $X = \{x_1, x_2, \ldots, x_n\}, x_i \in R^d$
- minimizes the following **fuzzy objective function**:

$$J_m = \sum_{i=1}^{n} \sum_{j=1}^{c} u_{ij}^{m} \parallel x_i - v_j \parallel^2$$

- n: number of data points
- c: number of clusters
- u_{ij}: membership of x_i cluster $j (0 \leq \mu_j \leq 1)$
- v_j: centroid of cluster j
- m: fuzziness coefficient (usually m=2) — controls "softness" of clustering
- $d_i = \|x_i - v_j\|$: Euclidean distance

FCM Algorithm Steps

- **Constraints**

$$\sum_{j=1}^{c} u_{ij} = 1 \qquad \forall i$$

2. Cluster Center Update

$$c_j = \frac{\sum_{i=1}^{n} u_{ij}^{m} x_i}{\sum_{i=1}^{n} u_{ij}^{m}}$$

3. Membership Update

$$u_{ij} = \frac{1}{\sum_{k=1}^{c} \left(\frac{\|x_i - v_j\|}{\|x_i - v_k\|} \right)^{\frac{2}{m-1}}}$$

FCM Algorithm Steps

4. Check for convergence:

- If change in U (or c_j) $<$ ε (a small threshold), stop.
- Otherwise, go back to step 2.

5. Stopping Condition

- Repeat steps 3–4 until: where ϵ is a small threshold (e.g., 10^{-5}).

$$\left\| \mathrm{V}^{(t+1)} - \mathrm{V}^{(t)} \right\| < \epsilon$$

Example: FCM Algorithm Steps

2.11##Perform a simple Fuzzy C-Means (FCM) clustering with 2 clusters and 3 data points in 1D.

Let's cluster 3 data points $x = \{1,2,5\}$ into number of clusters $C = 2$.

- Fuzziness parameter $m = 2$ (common choice)

1. Initialize membership matrix $U(Each\ row\ sums\ to\ 1)$:

$$U = \begin{bmatrix} 0.6 & 0.4 \\ 0.5 & 0.5 \\ 0.3 & 0.7 \end{bmatrix}$$

Example: FCM Algorithm Steps

3 data points $x = \{1,2,5\}$

1. Initialize membership matrix $U(Each\ row\ sums\ to\ 1)$:

$$U = \begin{bmatrix} 0.6 & 0.4 \\ 0.5 & 0.5 \\ 0.3 & 0.7 \end{bmatrix}$$

2. Compute cluster centers for $m = 2$:

- Calculate for cluster 1:

$$c_1 = \frac{0.6^2 \times 1 + 0.5^2 \times 2 + 0.3^2 \times 5}{0.6^2 + 0.5^2 + 0.3^2} = 1.87$$

- For cluster 2:

$$c_2 = \frac{0.4^2 \times 1 + 0.5^2 \times 2 + 0.7^2 \times 5}{0.4^2 + 0.5^2 + 0.7^2} = 3.46$$

Example: FCM Algorithm Steps

3. Update membership values u_{ij} using the distance-based formula (iteratively).

$$c_1 = 1.87, c_2 = 3.46$$

Since $\boldsymbol{m = 2}$, exponent $\frac{\mathbf{2}}{\boldsymbol{m-1}} = \mathbf{2}$. Thus,

$$d_i = \|x_j - c_i\|, \qquad d_k = \|x_j - c_k\|$$

$$u_{ij} = \frac{1}{\sum_{k=1}^{c} \left(\frac{\|x_j - c_i\|}{\|x_j - c_k\|} \right)^{\frac{2}{m-1}}} = \frac{1}{1 + \left(\frac{d_i}{d_j} \right)^2} \quad and$$

$$u_{ij+1} = 1 - u_{ij}$$

Example: FCM Algorithm Steps

For $x_1 = 1$: Calculate distances:

$$d_1 = |\ 1 - 1.87\ | = 0.87, \qquad d_2 = |\ 1 - 3.46\ | = 2.46$$

Calculate membership u_{11}:

$$u_{11} = \frac{1}{1 + \left(\frac{d_1}{d_2}\right)^2} = \frac{1}{1 + \left(\frac{0.87}{2.46}\right)^2} = 0.89$$

$$u_{12} = 1 - u_{11} = 1 - 0.89 = 0.11$$

Example: FCM Algorithm Steps

For $x_2 = 2$: Calculate distances:

$$d_1 = |\ 2 - 1.87\ | = 0.13, \qquad d_2 = |\ 2 - 3.46\ | = 1.46$$

Calculate membership u_{21}:

$$u_{11} = \frac{1}{1 + \left(\frac{d_1}{d_2}\right)^2} = \frac{1}{1 + \left(\frac{0.13}{1.46}\right)^2} = 0.992$$

$$u_{22} = 1 - u_{21} = 1 - 0.992 = 0.008$$

Example: FCM Algorithm Steps

For $x_3 = 5$: Calculate distances:

$$d_1 = |\ 5 - 1.87\ | = 3.13, \qquad d_2 = |\ 5 - 3.46\ | = 1.54$$

Calculate membership u_{31}:

$$u_{31} = \frac{1}{1 + \left(\frac{d_1}{d_2}\right)^2} = \frac{1}{1 + \left(\frac{3.13}{1.54}\right)^2} = 0.195$$

$$u_{32} = 1 - u_{31} = 1 - 0.195 = 0.805$$

Example: FCM Algorithm Steps

- **Updated Membership Matrix:**

$$U^{(1)} = \begin{bmatrix} 0.89 & 0.11 \\ 0.992 & 0.008 \\ 0.195 & 0.805 \end{bmatrix}$$

Calculate for cluster 1:

$$c_1' = \frac{0.89^2 \times 1 + 0.992^2 \times 2 + 0.195^2 \times 5}{0.89^2 + 0.992^2 + 0.195^2} = 1.63$$

For cluster 2:

$$c_2' = \frac{0.11^2 \times 1 + 0.008^2 \times 2 + 0.805^2 \times 5}{0.11^2 + 0.008^2 + 0.805^2} = 4.93$$

After a few more iterations, these values will stabilize.

Example: FCM Algorithm Steps

x	1	2	5
d	$d_1 = \lvert 1 - 1.63 \rvert = 0.63$ $d_2 = \lvert 1 - 4.93 \rvert = 3.93$	$d_1 = \lvert 2 - 1.63 \rvert = 0.37$ $d_2 = \lvert 2 - 4.93 \rvert = 2.93$	$d_1 = \lvert 5 - 1.63 \rvert = 3.37$ $d_2 = \lvert 5 - 4.93 \rvert = 0.07$
u_{ij}	$u_{13} = \frac{1}{1 + \left(\frac{0.63}{3.93}\right)^2} = 0.975$	$u_{23} = \frac{1}{1 + \left(\frac{0.37}{2.93}\right)^2} = 0.984$	$u_{33} = \frac{1}{1 + \left(\frac{3.37}{0.07}\right)^2} = 0.0$
u_{ij+1}	$u_{14} = 1 - u_{13} = 0.025$	$u_{24} = 1 - u_{23} = 0.016$	$u_{34} = 1 - u_{33} = 1$

Example: FCM Algorithm Steps

x	**1**	**2**	**5**
$U^{(2)}$	$U^{(2)} = \begin{bmatrix} 0.975 & 0.025 \\ 0.984 & 0.016 \\ 0.0 & 1.0 \end{bmatrix}$		
Cluster c	$c_1' = \frac{0.975^2 \times 1 + 0.984^2 \times 2 + 0.0^2 \times 5}{0.975^2 + 0.984^2 + 0.0^2} = 1.5$ $c_2' = \frac{0.025^2 \times 1 + 0.016^2 \times 2 + 1.0^2 \times 5}{0.025^2 + 0.016^2 + 1.0^2} = 4.99$		

The center of data point 1 and 2 indicate the around the center mean 1.5 and another center is 4.99 which is closed to data point 5 only. Thus, the algorithm grouped into

Cluster 1 → {1,2} and **Cluster 2** → {5}

Fuzzy C-Means (FCM) Clustering

- $x = \{1,2,5\}$, Number of clusters $c = 2$
- Fuzziness parameter $m = 2$ (common choice)

Step 1: Initialize cluster centers (randomly), Let's choose:

$$v_1 = 1 \; and \; v_2 = 5$$

Step 2: Calculate membership values uij

- For each data point x_j and cluster i, membership u_{ij} is:

$$u_{ij} = \frac{1}{\sum_{k=1}^{c} \left(\frac{\|x_j - v_i\|}{\|x_j - v_k\|} \right)^{\frac{2}{m-1}}}$$

- Since $m = 2$, the exponent is $2/(2-1) = 2$.

Fuzzy C-Means (FCM) Clustering

For $x_1 = 1$: Calculate distances:

$$d_{11} = | 1 - 1 | = 0$$
$$d_{12} = | 1 - 5 | = 4$$

But if $d_{ij} = 0$, then membership $u_{ij} = 1$ for that cluster and 0 for others (point exactly at cluster center).

Thus,

$$u_{11} = 1, u_{12} = 0$$

For $x_2 = 2$: Distances:

$$d_{21} = | 2 - 1 | = 1$$
$$d_{22} = | 2 - 5 | = 3$$

Fuzzy C-Means (FCM) Clustering

Calculate membership u_{21}:

$$u_{21} = \frac{1}{\left(\frac{d_{21}}{d_{21}}\right)^2 + \left(\frac{d_{21}}{d_{22}}\right)^2} = \frac{1}{1 + \left(\frac{1}{3}\right)^2} = 0.9$$

Similarly, u_{22}:

$$u_{21} = \frac{1}{\left(\frac{d_{22}}{d_{21}}\right)^2 + \left(\frac{d_{22}}{d_{22}}\right)^2} = \frac{1}{\left(\frac{3}{1}\right)^2 + 1} = 0.1$$

For $x_3 = 5$: Distances:

$d_{31} = \mid 5 - 1 \mid = 4$

$d_{32} = \mid 5 - 5 \mid = 0$

Membership:

$u_{31} = 0, u_{32} = 1$

Fuzzy C-Means (FCM) Clustering

Update cluster centers

New centers:

$$v_i = \frac{\sum_{j=1}^{n} u_{ij}^{m} x_j}{\sum_{j=1}^{n} u_{ij}^{m}}$$

Calculate for cluster 1:

$$v_1 = \frac{1^2 \times 1 + 0.9^2 \times 2 + 0^2 \times 5}{1^2 + 0.9^2 + 0^2} = \frac{1 + 1.62 + 0}{1 + 0.81 + 0} = 1.45$$

For cluster 2:

$$v_2 = \frac{0^2 \times 1 + 0.1^2 \times 2 + 1^2 \times 5}{0 + 0.01 + 1} = 4.97$$

Fuzzy C-Means (FCM) Clustering

Repeat steps 2 & 3 until convergence

Recalculate membership values using new centers

$$v_1 = 1.45, v_2 = 4.97$$

Then update centers again.

Repeat until cluster centers stabilize

Summary:

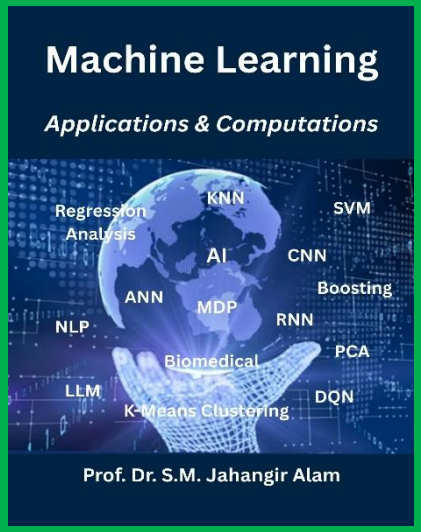
Point	Membership to Cluster 1	Membership to Cluster 2
1	1.00	0.00
2	0.90	0.10
5	0.00	1.00

Thank You

Lecture 3.5

Unsupervised Machine Learning

Gaussian Mixture Model (GMM), t-SNE, Hierarchical Clustering



Instructor Name:

Professor Dr. Jahangir Alam

Professor, Dept. of CSE, BAUST

Gaussian Mixture Model (GMM)

- A GMM is a probabilistic model that assumes data points are generated from a mixture of several Gaussian (Normal) distributions.
- It assumes data points are generated from a mixture of several Gaussian distributions.
- Each data point belongs to each Gaussian with some probability (soft assignment).

Gaussian Mixture Model (GMM)

- EM algorithm estimates parameters (means, variances, weights).
- **GMM**: clustering with soft probabilities, modeled as a mixture of Gaussians.
- Solved with **Expectation-Maximization (EM)**.
- More flexible than k-means.

Feature	K-Means	GMM
Cluster shape	Spherical	Elliptical (any covariance)
Assignment	Hard (belongs to 1 cluster)	Soft (probabilities)
Parameters	Centroids only	Mean + covariance + weight
Flexibility	Limited	More flexible, fits real-world data

Gaussian Mixture Model (GMM)

$$GMM:\ p(x) = \sum_{k=1}^{K} \pi_k \left(\mathrm{N}(x \mid \mu_k, \sum k \right)$$

where:

- K = number of Gaussians (clusters).
- π_k = mixing coefficient (weight of Gaussian k, with $\sum \pi_k = 1$).
- μ_k = mean of Gaussian k.
- $\sum k$ = covariance matrix of Gaussian k.
- $\mathrm{N}(x \mid \mu_k, \sum k)$ = Gaussian distribution.

Steps in GMM (using EM algorithm)

1. **Initialization**:

   Choose initial values for means (μ_k), covariances ($\sum k$), and mixing weights (π_k).

2. **E-step (Expectation):**

   Compute the responsibility (soft assignment): probability that data point x_i belongs to cluster k.

$$r_{ik} = \frac{\pi_k \mathrm{N}(x \mid \mu_k, \sum k)}{\sum_{j=1}^{K} \pi_j \mathrm{N}(x \mid \mu_j, \sum j)}$$

Steps in GMM (using EM algorithm)

3. **M-step (Maximization):** Update parameters using responsibilities:

Mixing coefficients	$\pi_k = \frac{1}{N}\sum_{i=1}^{N} r_{ik}$
Means	$\mu_k = \frac{\sum_{i=1}^{N} r_{ik} x_i}{\sum_{i=1}^{N} r_{ik}}$
Covariances	$\Sigma k = \frac{\sum_{i=1}^{N} r_{ik} (x_i - \mu_k)(x_i - \mu_k)^T}{\sum_{i=1}^{N} r_{ik}}$

Steps in GMM (using EM algorithm)

4. **Repeat *E* and *M*** until convergence.

convergence (**Convergence** means that an **iterative algorithm** (like gradient descent, EM, k-means, etc.) **stabilizes** after repeated updates, i.e. the values it is computing (parameters, loss, or likelihood) stop changing significantly.

The point where an iterative algorithm stabilizes (parameters or objective no longer change significantly).

Applications of GMM

Domain	Example
Image Segmentation	Segment MRI, CT, satellite images with overlapping regions
Speech Recognition	Model phoneme distributions
Finance	Model mixed distributions of returns, risk clustering
Anomaly Detection	Detect low-probability points as outliers
Biology	Gene expression clustering

Gaussian Mixture Model (GMM)

$x = \{1,2,5\}$; Number of components: $K = 2$

Initialize parameters: Let's guess:

- Means: $\mu_1 = 1, \mu_2 = 5$; Variances: $\sigma_1^2 = 1, \sigma_2^2 = 1$
- Mixing coefficients: $\pi_1 = 0.5, \pi_2 = 0.5$

E-step (Calculate responsibilities), For $x_1 = 1$:

$$\mathcal{N}(1 \mid 1,1) = \frac{1}{\sqrt{2\pi}} e^{-\frac{(1-1)^2}{2}} = 0.3989;\ \mathcal{N}(1 \mid 5,1) \approx 0.000134$$

Responsibilities:

$$r_{11} = \frac{0.5 \times 0.3989}{0.5 \times 0.3989 + 0.5 \times 0.000134} \approx 0.9997;$$

$$r_{12} = 1 - r_{11} = 0.0003$$

Gaussian Mixture Model (GMM)

Initialize parameters: Let's guess:

- Means: $\mu_1 = 1, \mu_2 = 5$; Variances: $\sigma_1^2 = 1, \sigma_2^2 = 1$
- Mixing coefficients: $\pi_1 = 0.5, \pi_2 = 0.5$

E-step (Calculate responsibilities), For $x_2 = 2$:

$$\mathcal{N}(2 \mid 1, 1) = 0.3989 \times e^{-\frac{(2-1)^2}{2}} = 0.24197$$

$$\mathcal{N}(2 \mid 5, 1) = 0.00443$$

$$r_{21} = \frac{0.5 \times 0.24197}{0.5 \times 0.24197 + 0.5 \times 0.00443} \approx 0.982$$

$$r_{22} = 1 - r_{21} = 0.018$$

Gaussian Mixture Model (GMM)

Initialize parameters: Let's guess:

- Means: $\mu_1 = 1, \mu_2 = 5$; Variances: $\sigma_1^2 = 1, \sigma_2^2 = 1$
- Mixing coefficients: $\pi_1 = 0.5, \pi_2 = 0.5$

E-step (Calculate responsibilities), For $x_3 = 5$:

$$\mathcal{N}(5 \mid 1, 1) = 0.3989 \times e^{-\frac{(5-1)^2}{2}} = 0.000134$$

$$\mathcal{N}(5 \mid 5, 1) = 0.3989$$

$$r_{31} = \frac{0.5 \times 0.000134}{0.5 \times 0.000134 + 0.5 \times 0.3989} \approx 0.0003$$

$$r_{32} = 1 - r_{31} = 0.9997$$

Gaussian Mixture Model (GMM)

M-step (Update parameters)

$$N_k = \sum_i r_{ik}$$

$$N_1 = 0.9997 + 0.982 + 0.0003 = 1.982$$

$$N_2 = 0.0003 + 0.018 + 0.9997 = 1.018$$

$$\mu_1 = \frac{0.9997 \times 1 + 0.982 \times 2 + 0.0003 \times 5}{1.982} = 1.496$$

$$\mu_2 = \frac{0.0003 \times 1 + 0.018 \times 2 + 0.9997 \times 5}{1.018} = 4.946$$

$$\sigma_1^2 = \frac{0.9997(1-1.496)^2 + 0.982(2-1.496)^2 + 0.0003(5-1.496)^2}{1.982} = 0.252$$

Gaussian Mixture Model (GMM)

$$\sigma_1^2 = \frac{0.9997(1-1.496)^2+0.982(2-1.496)^2+0.0003(5-1.496)^2}{1.982} = 0.252$$

$$\sigma_2^2 = 0.161$$

Update mixing coefficients:

$$\pi_k = \frac{N_k}{N} = \frac{N_k}{3}$$

$$\pi_1 = \frac{1.982}{3} = 0.66,$$

$$\pi_2 = \frac{1.018}{3} = 0.34$$

Summary after 1 iteration:

Component	μ_k	σ_k^2	π_k
1	1.496	0.252	0.66
2	4.946	0.161	0.34

Hierarchical Clustering

HC is a **clustering method** that builds a hierarchy (tree-like structure) of clusters. Unlike k-means, it doesn't require you to pre-specify the number of clusters; you can cut the hierarchy at any level to choose the number you want.

Two Main Types

- **Agglomerative (bottom-up):**
  - Start with each point as its own cluster.
  - Iteratively **merge** the two closest clusters.
  - Continue until only one cluster remains.
  - This is the most common.

- **Divisive (top-down):**
  - Start with all points in one big cluster.
  - Recursively **split** clusters.
  - Continue until each point is its own cluster.

Hierarchical Clustering

Steps of Agglomerative Clustering

- Compute **distance matrix** (e.g., Euclidean distance between points).
- Initially, treat each point as a cluster.
- Merge the two clusters with the smallest distance.
- Update the distance matrix to reflect the new cluster distances:
  - **Single linkage:** distance = minimum pairwise distance between points in two clusters.
  - **Complete linkage:** distance = maximum pairwise distance.
  - **Average linkage:** distance = average of all pairwise distances.
  - **Centroid linkage:** distance between centroids.
- Repeat steps 3–4 until one cluster remains.
- The process is typically visualized using a **dendrogram**.

Applications of Hierarchical Clustering

Domain	Application
Bioinformatics	Gene expression clustering, phylogenetic trees
Marketing	Customer segmentation
Document Clustering	News/articles grouping by topic
Image Processing	Segmenting pixels into regions
Social Networks	Detecting communities

Hierarchical Clustering

- **Data points (1D for simplicity):**

$$A = 2, B = 5, C = 8, D = 10$$

Calculate the distance matrix

Using **Euclidean distance** (absolute difference since 1D):

	A=2	B=5	C=8	D=10
A=2	0	3	6	8
B=5	3	0	3	5
C=8	6	3	0	2
D=10	8	5	2	0

Find the closest pair:

Minimum distance = 2 between points **C and D**

Hierarchical Clustering

	A=2	B=5	C=8	D=10
A=2	0	3	6	8
B=5	3	0	3	5
C=8	6	3	0	2
D=10	8	5	2	0

- **Merge clusters C and D**
- New cluster: $CD = \{8,10\}$

	A=2	B=5	CD={8,10}
A=2	0	3	6
B=5	3	0	3
CD	6	3	0

New cluster: AB={2,5}

	AB={2,5}	CD={8,10}
AB	0	3
CD	3	0

Merge clusters AB and CD

Distance = 3

Final cluster contains all points {2,5,8,10}

The merge order (dendrogram):

1. Merge C and D at distance 2
2. Merge A and B at distance 3
3. Merge AB & CD at distance 3

t-SNE

t-SNE (t-Distributed Stochastic Neighbor Embedding) is a **non-linear dimensionality reduction** and **visualization** technique.

It is especially useful for visualizing **high-dimensional data** (like images, text embeddings, gene expression) in **2D or 3D**.

t-SNE tries to **preserve local structure** (neighborhoods) of data when mapping from high dimensions to low dimensions.

t-SNE

- In high dimensions → Similar points should have high probability of being neighbors.
- Computing **pairwise similarities** in high-dimensional space using a **Gaussian** kernel.
- Mapping those similarities to low-dimensional space using a **Student *t*-distribution**.
- Minimizing the KL(**Kullback–Leibler**) divergence between both distributions.
- In low dimensions → The embedding tries to reproduce these probabilities.

How t-SNE Works?

1. **Compute pairwise similarities in high-dimensional space**

- For each pair of points x_i, x_j, compute conditional probability:

$$p_{j|i} = \frac{e^{-\frac{\|x_i - x_j\|^2}{2\sigma_i^2}}}{\sum_{k \neq i} e^{-\frac{\|x_i - x_k\|^2}{2\sigma_i^2}}}$$

where σ_i controls the neighborhood size (related to **perplexity (AI-powered Search & answer Engine)**). Then define:

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2N}$$

(symmetrized probability that x_i *and* x_j are neighbors).

t-SNE

2. **Compute pairwise similarities in low-dimensional space (y-space)**

- For mapped points y_i , y_j in 2D/3D, use a **Student-t distribution** with 1 degree of freedom (Cauchy distribution):

$$q_{ij} = \frac{(1+\| y_i - y_j \|^2)^{-1}}{\sum_{k \neq l}(1+\| y_k - y_l \|^2)^{-1}}$$

This heavy-tailed distribution avoids the **crowding problem**.

3. **Match the distributions (minimize KL divergence)**

- Minimize the difference between pijp_{ij}pij and qijq_{ij}qij using **Kullback–Leibler (KL) divergence**:

$$C = KL(P \parallel Q) = \sum_{i \neq j} p_{ij} log \frac{p_{ij}}{q_{ij}}$$

Optimize using **gradient descent**.

t-SNE

Key Parameters:

- **Perplexity**: Controls neighborhood size (typical range 5–50).
- **Learning rate**: Affects convergence.
- **n_iter**: Number of iterations (≥1000 recommended).

Applications

- Visualizing word embeddings (NLP)
- Visualizing MNIST or CIFAR images
- Gene expression clustering (bioinformatics)
- High-dimensional customer segmentation

Example: t-SNE

Point	x	y
A	0	0
B	1	0
C	5	0

This setup means A and B are close, and C is far from both.

Step 1: Compute Euclidean distances (high-dimensional space)

$$d(A,B) = \sqrt{(1-0)^2 + (0-0)^2} = 1$$

$$d(A,C) = \sqrt{(5-0)^2} = 5\,,\; d(B,C) = \sqrt{(5-1)^2} = 4$$

Step 2: Compute similarities (conditional probability $p_{j/i}$)

Using a **Gaussian kernel** (simplified, symmetric):

$$p_{j/i} = \frac{e^{-d_{ij}^2/2\sigma^2}}{\sum_{k \neq i} e^{-d_{ik}^2/2\sigma^2}}$$

Let, $\sigma = 1$. Now compute: For **A**:

$$p_{B/A} = \frac{e^{-1^2/2}}{e^{-1^2/2} + e^{-5^2/2}}$$

$$= \frac{0.6065}{0.6065 + 0.0000037} \approx 0.999994$$

$$p_{C/A} \approx 0.000006$$

Example: t-SNE

Point	x	y
A	0	0
B	1	0
C	5	0

This setup means A and B are close, and C is far from both.

$$p_{AB} = \frac{p_{B|A} + p_{A|B}}{2} \approx \frac{0.999994 + 0.999664}{2} \approx 0.9998$$

$$p_{AC} \approx \frac{0.000006 + 0.0179}{2} \approx 0.009$$

$$p_{BC} \approx \frac{0.000336 + 0.9821}{2} \approx 0.491$$

Example: t-SNE

Point	x	y
A	0	0
B	1	0
C	5	0

Step 3: Initialize low-dimensional (2D) points randomly

Let's start with: A' = (-1, 0), B' = (0, 0)

C' = (2, 0)

Step 4: Compute low-dimensional similarities $q_{i/j}$

Using **Student-t kernel** (with 1 degree of freedom):

$$q_{i/j} = \frac{\left(1+\|y_i - y_j\|^2\right)^{-1}}{\sum_{k \neq l}(1+\|y_k - y_l\|^2)^{-1}}$$

Compute distances:

- $||A' - B'|| = 1$
- $||A' - C'|| = 3$
- $||B' - C'|| = 2$

- Numerator: $(1 + 1^2)^1 = \frac{1}{2} = 0.5$
- $(1 + 3^2)^1 = \frac{1}{10} = 0.1$
- $(1 + 2^2)^1 = \frac{1}{5} = 0.2$

Normalize: $Z = 0.5 + 0.1 + 0.2 = 0.8$

$q_{AB} = 0.5/0.8 = 0.625, q_{AC} = 0.1/0.8 = 0.125, q_{BC} = 0.2/0.8 = 0.25$

Example: t-SNE

- **Step 5: Compare p_{ij} and q_{ij}**

Pair	p_{ij}	q_{ij}	High match?
AB	0.9998	0.625	too low
AC	0.009	0.125	too high
BC	0.491	0.25	too low

- → The KL divergence is high, so we would **update the 2D positions** (e.g., move A′ and B′ closer, push A′ and C′ apart) to reduce the mismatch.

t-SNE tries to do:

- Make **similar points** in high-D space **stay close** in low-D.
- Make **dissimilar points** be **far apart**.
- Use **gradient descent** to iteratively update positions.

Summary: t-SNE

- We computed pairwise distances.
- Transformed them into high-dimensional and low-dimensional similarities.
- Compared how well the low-D representation reflects the original.
- In practice, t-SNE would **optimize** the layout over 100 of iterations.

Thank You